DEVELOPMENT OF A SOFTWARE APPLICATION FOR ANALYSIS AND PROCESSING OF DVB-S2 RECEIVER OUTPUT STREAM

A PROJECT REPORT
Submitted in Partial Fulfillment of the Requirements
for the Award of the Degree of
BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING

Submitted by

Y VENGALA RAO (Roll No. 20231COM0001)
SK SAMAD (Roll No. 20231COM0031)
C RAKESHWAR (Roll No. 20231COM0008)

Under the guidance of
Asst. Prof. IRFAN RAJAB BHAT
Assistant Professor, Department of Computer Science & Engineering

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING
PRESIDENCY UNIVERSITY
BENGALURU - 560064
NOVEMBER 2026

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING
PRESIDENCY UNIVERSITY, BENGALURU

BONAFIDE CERTIFICATE

Certified that this project report entitled "Development of a Software Application for Analysis and Processing of DVB-S2 Receiver Output Stream" is a bonafide work of Mr. Y VENGALA RAO (Roll No. 20231COM0001), Mr. SK SAMAD (Roll No. 20231COM0031), and Mr. C RAKESHWAR (Roll No. 20231COM0008), who have successfully carried out the project work under my supervision and submitted the report in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE AND ENGINEERING from Presidency University, Bengaluru during the academic year 2026-27.

The project work described in this report represents genuine scientific investigation and technical engineering. The software architecture, empirical evaluations, mathematical formulations, and verification test suites have not been submitted elsewhere for the award of any other degree or diploma.

Asst. Prof. IRFAN RAJAB BHAT **Project Guide, PSCS** **Presidency University**	**Ms. SUMITA GUDDIN** **Program Project Coordinator, PSCS** **Presidency University**	**Dr. GOPAL K SHYAM** **Head of the Department, PSCS** **Presidency University**

Date: 28-11-2026
Place: Bengaluru

Official Institutional Seal

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING
PRESIDENCY UNIVERSITY, BENGALURU

CERTIFICATE OF VIVA-VOCE EXAMINATION

This is to certify that Mr. Y VENGALA RAO (Roll No. 20231COM0001), Mr. SK SAMAD (Roll No. 20231COM0031), and Mr. C RAKESHWAR (Roll No. 20231COM0008) have been subjected to the Viva-Voce Examination on 28-11-2026 at Presidency University, Bengaluru, for their project report titled "Development of a Software Application for Analysis and Processing of DVB-S2 Receiver Output Stream" in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering.

The project performance, technical demonstrations, architectural defenses, and comprehensive documentation presented by the candidates have been evaluated and approved by the examination committee.

_____	_____
Internal Examiner	**External Examiner**
Name: Asst. Prof. IRFAN RAJAB BHAT	**Name: _____**
Designation: Assistant Professor	**Designation: _____**
Presidency University, Bengaluru	**University/Industry: _____**

PRESIDENCY UNIVERSITY
PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

We, the students of final year B.Tech in COMPUTER SCIENCE AND ENGINEERING at Presidency University, Bengaluru, named Y VENGALA RAO (Roll No. 20231COM0001), SK SAMAD (Roll No. 20231COM0031), and C RAKESHWAR (Roll No. 20231COM0008), hereby declare that the project work titled "Development of a Software Application for Analysis and Processing of DVB-S2 Receiver Output Stream" has been independently carried out by us and submitted in partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science and Engineering during the academic year 2026-27.

We further confirm that the software implementation, empirical results, mathematical evaluations, and written prose comprising this report are our original creations. External technical literature, standards documents, and libraries have been properly referenced and cited in accordance with academic integrity guidelines. This work has not been submitted previously to any other university or institution for any academic degree, diploma, or certificate.

Y VENGALA RAO	**SK SAMAD**	**C RAKESHWAR**
(20231COM0001)	**(20231COM0031)**	**(20231COM0008)**

Date: 28-11-2026
Place: Bengaluru

ACKNOWLEDGEMENTS

We express our deepest gratitude to the Chancellor, Vice-Chancellor, and Pro Vice-Chancellor of Presidency University, Bengaluru, for providing the state-of-the-art computational infrastructure, research laboratories, and an inspiring academic environment that enabled the successful execution of this capstone project.

We place on record our sincere appreciation and respectful gratitude to Dr. GOPAL K SHYAM, Head of the Department of Computer Science and Engineering, for his constant encouragement, visionary leadership, and persistent focus on technical rigor and research excellence across our engineering program.

We are profoundly indebted to our project guide, Asst. Prof. IRFAN RAJAB BHAT, Assistant Professor, Department of Computer Science and Engineering, for his exceptional guidance, intellectual mentorship, and critical technical feedback throughout the design, algorithmic formulation, and empirical testing phases of this project. His high standards for academic integrity and systems engineering shaped the depth and precision of this work.

We also convey our sincere thanks to Ms. SUMITA GUDDIN, Program Project Coordinator, and all faculty members and technical staff of the Presidency School of Computer Science and Engineering for their continuous coordination, administrative support, and helpful reviews during our milestone presentations.

Finally, we extend our heartfelt appreciation to our parents, family members, and fellow students for their unwavering moral support, encouragement, and understanding throughout the course of our B.Tech engineering studies.

Y VENGALA RAO (20231COM0001)
SK SAMAD (20231COM0031)
C RAKESHWAR (20231COM0008)

ABSTRACT

Second Generation Digital Video Broadcasting by Satellite (DVB-S2, ETSI EN 302 307) serves as the primary telecommunications transmission standard underpinning global satellite broadband connectivity, cellular backhaul, and high-definition direct-to-home broadcasting. Post-demodulator digital output emitted by modern commercial and defense satellite receiver hardware comprises structurally diverse container encapsulation formats, notably fixed 188-byte MPEG-2 Transport Streams (MPEG-TS), variable-length Generic Stream Encapsulation (GSE) protocol data units, and native physical-layer Baseband Frames (BBFrame). Assessing stream integrity, identifying multiplex irregularities, and isolating frame corruption downstream of the demodulation stage present acute operational challenges in real-world ground station environments. Existing diagnostic workflows are severely constrained by rigid static thresholding in commercial broadcast probes, total diagnostic blind spots regarding multi-format container fragmentation, unwarranted physical-layer extrapolations, and the absence of unsupervised anomaly detection capabilities.

This capstone project presents PRJ_111, a comprehensive, self-contained software workstation engineered for deterministic stream health accounting, unsupervised anomaly detection, and automated diagnostic synthesis across multi-format DVB-S2 receiver output streams. The software delivers an integrated seven-feature analytical pipeline (F1 through F7) incorporating: deterministic stream health scoring anchored in ETSI TR 101 290 Priority-1 measurement principles; unsupervised outlier detection via Isolation Forest ensembles operating across 8-dimensional sliding window feature spaces; byte-exact spatial timeline visualizations referencing physical bitstream offsets (0x...); comprehensive continuity counter tracking; explainable machine learning diagnoses using clamped Z-score feature attributions (|Z| <= 20 sigma); cross-format differential stream comparisons governed by an automated semantic barrier; and multi-format report synthesis across JSON, Markdown, Text, and standalone HTML formats.

The software architecture incorporates a chunked streaming reader processing multi-gigabyte bitstreams within 64 KB memory-bounded buffers, an empirical fault-injection benchmark utility, and an offline single-page application (SPA) operator dashboard with zero external

cloud dependencies. Empirical evaluation across physical satellite bitstreams confirms 100% parsing accuracy: 18,176 packets in MPEG-TS (3,267,305 bytes payload, health score 100.0), 14 PDUs in GSE (57.1% fragmentation, 8,764 bytes payload), and 4,309 frames in BBFrame (100% ACM, modal 2,992-bit DFL, 100% valid CRC-8). The complete automated regression test suite comprising 304 tests across 18 modules executes with a 100% pass rate in 76.06 seconds, establishing a reproducible, verifiable, and highly scalable software workstation for satellite ground station telemetry monitoring.

Keywords: *DVB-S2, MPEG-TS, Generic Stream Encapsulation (GSE), BBFrame, Isolation Forest, ETSI TR 101 290, Stream Health Analysis, Diagnostic Reporting, Chunked Processing.*

TABLE OF CONTENTS

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	ABSTRACT	vi
	LIST OF FIGURES	ix
	LIST OF TABLES	x
	LIST OF ABBREVIATIONS	xi
CHAPTER 1	INTRODUCTION	1
1.1	Background and Context of Satellite Digital Broadcasting	1
1.2	Digital Demodulation Output Stream Paradigm	2
1.3	Problem Statement	3
1.4	Project Objectives	4
1.5	Scope of the Work	5
1.6	Motivation and Industrial Significance	6
1.7	Organization of the Report	7
CHAPTER 2	LITERATURE SURVEY	8
2.1	Evolution of Satellite Transmission Standards	8
2.2	ETSI TR 101 290 Priority-1 Measurement Principles	9
2.3	General-Purpose Network Protocol Analyzers (Wireshark / tcpdump)	10
2.4	Dedicated Broadcast Transport Stream Analyzers	11
2.5	Machine Learning in Telemetry Anomaly Detection	12
2.6	Explainable AI (XAI) in Diagnostic Monitoring	13
2.7	Streaming Bitstream Processing and Chunked Ingestion Paradigms	14
2.8	Synthesis of Literature and Comparative Framework	15

CHAPTER NO	TITLE	PAGE NO
CHAPTER 3	RESEARCH GAPS OF EXISTING METHODS	16
3.1	Rigid Static Thresholding and Lack of Dynamic Behavioral Modeling	16
3.2	Container Fragmentation and Multi-Format Inflexibility	17
3.3	Artificial Timestamp Inflation vs. Physical Offset Traceability	18
3.4	Unwarranted Cross-Format Telemetry Bleed and Semantic Incoherence	19
3.5	High Acquisition Cost and Proprietary Lock-in of Dedicated Probes	20
3.6	Diagnostic Opacity and Lack of Root-Cause Explanations	21
3.7	Comprehensive Comparative Matrix	22
CHAPTER 4	PROPOSED METHODOLOGY	23
4.1	Foundational Architecture Principle: Container Abstraction	23
4.2	Unified Multi-Format Feature Extraction Pipeline	24
4.3	Mathematical Formulations and Analytical Models	25
4.4	Chunked Streaming Processing Engine (64 KB Bounded Buffer)	28
4.5	Controlled Fault-Injection Validation Methodology	30
CHAPTER 5	OBJECTIVES AND SPECIFICATIONS	32
5.1	Primary Functional Objectives (F1 through F7)	32
5.2	Non-Functional and Operational Requirements	34
5.3	Mathematical and Diagnostic Verification Targets	35
5.4	Automated Test Suite Coverage Targets	36
CHAPTER 6	SYSTEM DESIGN AND IMPLEMENTATION	37
6.1	Architectural Overview and Layer Separation	37
6.2	Low-Level Container Parsing Subsystem	38
6.3	Unified Feature Extraction Subsystem	41

CHAPTER NO	TITLE	PAGE NO
6.4	Feature F1: Stream Health & Priority-1 Analysis	42
6.5	Feature F2: Unsupervised Isolation Forest Anomaly Detection	43
6.6	Feature F3: Structural Pattern Detection & Multiplex Dynamics	44
6.7	Feature F4: Spatial-Offset Timeline & Burst Localization	45
6.8	Feature F5: Explainable Anomaly Root-Cause Diagnostic Engine	46
6.9	Feature F6: Dual-Stream Comparison Engine with Semantic Barrier	47
6.10	Feature F7: Multi-Format Automated Diagnostic Report Synthesis	48
6.11	Frontend Engineering Workstation Architecture	49
6.12	Streaming Engine & High-Throughput Chunked Implementation	51
6.13	Controlled Fault-Injection Framework Implementation	52
CHAPTER 7	TIMELINE FOR EXECUTION OF PROJECT (GANTT CHART)	53
7.1	Five-Phase Project Milestone Execution Lifecycle	53
7.2	Detailed Milestone Breakdown (Reviews 0 to 3 / Final)	54
7.3	Milestone Deliverables and Timeline Schedule	55
CHAPTER 8	OUTCOMES	57
8.1	Tangible Software and Engineering Deliverables	57
8.2	Academic Research Outcomes and Technical Publications	58
8.3	Standard Operating Procedures and Reproducibility Protocols	59
CHAPTER 9	RESULTS AND DISCUSSIONS	60
9.1	Experimental Setup and Execution Environment	60
9.2	Verification of Complete Automated Test Suite (304 / 304 Passing)	61
9.3	Empirical Benchmark Evaluation Across Physical Satellite Bitstreams	63
9.4	Controlled Synthetic Fault Injection Sensitivity & Escalation Experiments	66

CHAPTER NO	TITLE	PAGE NO
9.5	Memory Footprint, Streaming Throughput & Complexity Analysis	68
9.6	Comparative Evaluation: PRJ_111 vs. Wireshark vs. Commercial Hardware	70
9.7	Discussion of Empirical Findings and Practical Engineering Takeaways	71
CHAPTER 10	CONCLUSION AND FUTURE WORK	72
10.1	Summary of Scientific and Engineering Contributions	72
10.2	Technical Limitations and Operational Boundaries	73
10.3	Future Research and Engineering Enhancements	74
	REFERENCES	75
APPENDIX A	PSEUDOCODE AND CORE ALGORITHMS	78
APPENDIX B	APPLICATION SCREENSHOTS AND ENGINEERING WORKSTATION	82
APPENDIX C	MANDATORY ENCLOSURES (Publication, Awards, Originality, SDGs)	86
APPENDIX D	VERIFICATION & REPRODUCIBILITY LEDGER (Hashes & Test Logs)	89

LIST OF FIGURES

LIST OF FIGURES

FIGURE NO	FIGURE CAPTION	PAGE NO
Figure 4.1	End-to-End Multi-Format System Architecture of PRJ_111	24
Figure 7.1	Five-Phase Project Milestone Execution Roadmap (Gantt Schedule)	56
Figure B.1	Standby Ingestion Dashboard Awaiting File Upload	82
Figure B.2	Live Dashboard Rendering Health Gauge and PID Distribution for sample.ts	82
Figure B.3	Unsupervised Isolation Forest Outlier Plot & Diagnostic Attribution View	83
Figure B.4	Spatial-Offset Timeline View Scrubbing Across Physical Byte Offsets	83
Figure B.5	Side-by-Side Dual-Stream Comparison View Governed by Semantic Shield	84
Figure B.6	Multi-Format Diagnostic Report Preview and Direct Download Modal	84
Figure B.7	Robust Input Validation Rejection Dialog for Corrupted Non-Stream File	85

LIST OF TABLES

LIST OF TABLES

TABLE NO	TABLE CAPTION	PAGE NO
Table 1.1	Target Input Formats and Physical Layer Demarcation	3
Table 2.1	Comparison of Related DVB Stream Analysis Utilities	11
Table 3.1	Objective Architectural Comparison: Wireshark vs. PRJ_111	22
Table 4.1	Unified Feature Vector Dimensions for Rolling Anomaly Detection	25
Table 4.2	Priority-1 Integrity Fault Identifiers, Trigger Conditions, and Penalty Weights	26
Table 5.1	Mapping of Functional Features F1 through F7 to Verified Architectural State	33
Table 6.1	Subsystem Components, Implementation Files, and Design Roles	38
Table 6.2	Semantic Cross-Format Comparability Matrix for Dual-Stream Analysis	48
Table 7.1	Milestone Execution Schedule and Engineering Deliverables	55
Table 8.1	Tangible Software Artifacts and Release Distribution	58
Table 9.1	Experimental Computing Environment and Dependency Specifications	61
Table 9.2	Complete Automated Test Suite Breakdown Across 18 Test Modules (304 Tests)	62
Table 9.3	Master Empirical Experimental Results Across Physical Satellite Bitstreams	64
Table 9.4	Controlled Synthetic Fault Injection Sensitivity and Detection Benchmark	67
Table 9.5	Chunked Ingestion Streaming Engine Performance and Throughput Benchmark	69
Table 9.6	Comprehensive Milestone Bug History and Engineering Resolutions	70
Table C.1	United Nations Sustainable Development Goals (SDG) Alignment Matrix	88
Table D.1	Cryptographic Checksum Ledger (SHA-256) of Raw Datasets and Core Artifacts	90

LIST OF ABBREVIATIONS

LIST OF ABBREVIATIONS

ABBREVIATION	EXPANSION / DEFINITION
ACM	Adaptive Coding and Modulation
AFC	Adaptation Field Control
API	Application Programming Interface
BBFrame	Baseband Frame (Physical layer container in DVB-S2)
BBHeader	Baseband Header (10-byte descriptor at the start of a BBFrame)
BER	Bit Error Rate
CC	Continuity Counter (4-bit field in MPEG-TS header)
CNR	Carrier-to-Noise Ratio
CRC	Cyclic Redundancy Check
DFL	Data Field Length (16-bit field in BBHeader indicating user payload bits)
DTH	Direct-to-Home (Satellite broadcasting service)
DVB	Digital Video Broadcasting
DVB-S2	Digital Video Broadcasting - Satellite - Second Generation (ETSI EN 302 307)
DVB-S2X	Digital Video Broadcasting - Satellite - Second Generation Extensions
ES6	ECMAScript 2015 (Modern JavaScript Standard)
ETSI	European Telecommunications Standards Institute
FEC	Forward Error Correction
GSE	Generic Stream Encapsulation (ETSI TS 102 606)
GUI	Graphical User Interface
HOD	Head of the Department
HTML5	Hypertext Markup Language Revision 5

ABBREVIATION	EXPANSION / DEFINITION
IEEE	Institute of Electrical and Electronics Engineers
IP	Internet Protocol
IQR	Interquartile Range
ISO	International Organization for Standardization
JSON	JavaScript Object Notation
LT	Length Type (2-bit protocol field in GSE PDU header)
MATYPE	Mode Adaptation Type (2-byte descriptor field in BBHeader)
ML	Machine Learning
MODCOD	Modulation and Coding Scheme
MPEG-TS	Moving Picture Experts Group - Transport Stream (ISO/IEC 13818-1)
MVP	Minimum Viable Product
PCAP	Packet Capture (Network bitstream capture file format)
PCR	Program Clock Reference
PDU	Protocol Data Unit
PID	Packet Identifier (13-bit header field in MPEG-TS)
PSCS	Presidency School of Computer Science and Engineering
PSI/SI	Program Specific Information / Service Information
PUSI	Payload Unit Start Indicator (1-bit flag in MPEG-TS header)
QoS	Quality of Service
REST	Representational State Transfer
RF	Radio Frequency
SDG	Sustainable Development Goal (United Nations)
SNDU	Subnetwork Dependent Convergence Protocol Network Data Unit
SPA	Single-Page Application

ABBREVIATION	EXPANSION / DEFINITION
SYNC	Synchronization Byte (0x47 in MPEG-TS)
TCP	Transmission Control Protocol
TEI	Transport Error Indicator (1-bit flag in MPEG-TS header)
TR	Technical Report (ETSI standard document series)
TS	Transport Stream
UPL	User Packet Length (16-bit field in BBHeader)
VCM	Variable Coding and Modulation
XAI	Explainable Artificial Intelligence

CHAPTER 1
INTRODUCTION

1.1 Background and Context of Satellite Digital Broadcasting

Modern telecommunications infrastructure relies extensively on satellite communications to deliver high-throughput broadband internet, direct-to-home (DTH) digital television, cellular backhaul, and tactical defense data links across vast geographical expanses. The cornerstone of contemporary satellite communications is the Second Generation Digital Video Broadcasting by Satellite standard (DVB-S2), formally codified by the European Telecommunications Standards Institute under ETSI EN 302 307. DVB-S2 achieved unprecedented spectral efficiency, operating within 0.7 dB to 1 dB of the theoretical Shannon capacity limit through the integration of advanced Forward Error Correction (FEC) codes—specifically Low-Density Parity-Check (LDPC) outer codes concatenated with Bose-Chaudhuri-Hocquenghem (BCH) inner codes—alongside flexible framing and higher-order modulations including QPSK, 8PSK, 16APSK, and 32APSK.

A transformative capability introduced in DVB-S2 is Adaptive Coding and Modulation (ACM) and Variable Coding and Modulation (VCM). Under ACM, physical transmission parameters—such as modulation constellation and FEC code rates—dynamically adapt on a frame-by-frame basis in response to instantaneous channel state telemetry and atmospheric fade conditions reported by receiving ground stations. While ACM optimizes link utilization across unpredictable satellite propagation channels, it creates extreme structural heterogeneity in the post-demodulated digital output streams emitted by receiving hardware. Modern ground station receivers must ingest, demultiplex, and validate bitstreams whose encapsulation formats, frame boundaries, and bit rates vary dynamically over time.

1.2 Digital Demodulation Output Stream Paradigm

In satellite receiver architecture, the physical receiving pipeline comprises three primary stages: the radio frequency (RF) front-end and tuner, the digital demodulator and FEC decoder, and the transport/network de-encapsulation processor. The physical demodulator

locks onto the modulated carrier, synchronizes carrier frequency and phase, computes log-likelihood ratios, and executes LDPC/BCH error correction. Once error correction is completed, the demodulator emits a digital bitstream. Depending on the operational profile and architectural deployment of the ground terminal, this post-demodulation output is presented in one of three mutually exclusive stream container formats:

- **MPEG-2 Transport Stream (MPEG-TS):** The legacy broadcast multiplex standard (ISO/IEC 13818-1) composed of rigid 188-byte fixed-length packets, each beginning with a synchronization byte (0x47). MPEG-TS encapsulates video, audio, and data elementary streams multiplexed via 13-bit Packet Identifiers (PIDs).
- **Generic Stream Encapsulation (GSE):** The modern IP-over-satellite encapsulation protocol (ETSI TS 102 606) engineered to transport native IPv4/IPv6 packets and Ethernet frames with low protocol overhead. GSE utilizes variable-length Protocol Data Units (PDUs) with dynamic fragmentation (Start/End flags).
- **DVB-S2 Baseband Frames (BBFrame):** The fundamental transmission container directly emitted by the physical-layer demodulator, consisting of a fixed 10-byte Baseband Header (BBHeader) followed by a payload data field (DFL bits). The BBHeader encodes transmission mode adaptation flags (MATYPE), user packet length (UPL), and an 8-bit Cyclic Redundancy Check (CRC-8).

> **ARCHITECTURAL INVARIANT: ALTERNATIVE INPUT FORMAT REALITY**
> A foundational engineering rule established in PRJ_111 is that BBFrame, GSE, and MPEG-TS represent alternative, mutually exclusive manifestations of the receiver output stream. A physical satellite bitstream captured at a specific digital interface point is either an MPEG-TS multiplex, a GSE PDU sequence, or raw baseband frames. The analyzer must auto-detect and isolate container semantics natively rather than attempting invalid multi-layer nested assumptions.

Table 1.1: Target Input Formats and Physical Layer Demarcation

Input Format	Standard Reference	Framing Mechanism	Key Telemetry Fields	Target Service Profile
MPEG-TS	ISO/IEC 13818-1	Fixed 188-byte packets (0x47 sync)	PID, Continuity Counter, TEI, PUSI, AFC	Direct-to-Home Broadcast, Legacy IPTV

Input Format	Standard Reference	Framing Mechanism	Key Telemetry Fields	Target Service Profile
GSE	ETSI TS 102 606	Variable-length PDUs (Frag flags)	Protocol Type, Total Length, Frag S/E, Label	Satellite Broadband, IP Backhaul, VSAT
BBFrame	ETSI EN 302 307	10-byte BBHeader + DFL payload	MATYPE, UPL, DFL, SYNC, SYNCD, CRC-8	Physical Demodulator Output, Raw Telemetry

1.3 Problem Statement

Satellite ground station operators, telecommunications engineers, and satellite ISP administrators frequently encounter subtle stream corruption, multiplex irregularities, and container degradation in post-demodulated digital streams. However, existing diagnostic tools and operational workflows exhibit critical deficiencies:

- **Format Fragmentation and Siloed Tools:** Legacy broadcast test equipment (e.g., dedicated hardware transport stream analyzers) is rigidly coupled to fixed 188-byte MPEG-TS and is incapable of ingesting or interpreting Generic Stream Encapsulation (GSE) or raw DVB-S2 Baseband Frames. Conversely, generic network analyzers such as Wireshark treat satellite bitstreams as opaque byte dumps without native awareness of DVB-S2 baseband header CRC-8 structures, multiplex entropy, or ETSI TR 101 290 Priority-1 integrity indicators.
- **Rigid Static Thresholding vs. Dynamic Streams:** Existing monitoring probes rely on rigid, hardcoded alarm thresholds (e.g., flag error if continuity counter gap exceeds 1). Under dynamic ACM/VCM switching and variable-rate IP traffic bursts, static thresholds trigger severe false alarm fatigue or completely fail to detect subtle, non-deterministic anomalies such as structural distribution drift, silent packet stuffing, and component starvation.
- **Lack of Byte-Exact Spatial Traceability:** When an anomaly occurs, operators must pinpoint the exact physical bitstream byte offset to inspect the corrupted frame. Existing

software tools inflate artificial timestamps or abstract away the raw byte coordinates, preventing deterministic correlation with physical storage.

- **Opaque Anomaly Classifications:** Contemporary machine learning prototypes that apply outlier detection typically output opaque binary labels or raw scalar anomaly scores without explainable feature attributions, leaving ground station engineers unable to deduce the underlying root cause.

1.4 Project Objectives

To resolve these operational and engineering challenges, the primary objective of this project is to develop, verify, and deliver PRJ_111: a self-contained, high-performance software application engineered for comprehensive analysis, deterministic health scoring, unsupervised anomaly detection, and automated diagnostic synthesis across multi-format DVB-S2 receiver output streams. The specific technical objectives are:

- **Multi-Format Container Parsing Subsystem:** Implement robust, zero-copy parsers capable of deterministic auto-detection and validation across MPEG-TS (188-byte fixed packets), GSE (variable-length PDUs with fragmentation), and DVB-S2 BBFrames (10-byte headers with bitwise CRC-8 verification).
- **Deterministic Health Scoring (Feature F1):** Implement an automated stream health assessment engine inspired by ETSI TR 101 290 Priority-1 measurement principles, computing a bounded health index [0, 100] reflecting sync acquisition stability, continuity counter integrity, and transport error indicators.
- **Unsupervised AI Anomaly Detection (Feature F2):** Integrate an unsupervised Isolation Forest machine learning engine operating over multi-dimensional sliding feature vectors to isolate structural and statistical multiplex anomalies without requiring pre-labeled training datasets.
- **Structural Multiplex Pattern Detection (Feature F3):** Develop analytical algorithms to compute Shannon multiplex component entropy, identify dominant PIDs, quantify GSE encapsulation efficiency, and extract modal BBFrame Data Field Lengths (DFL).

- **Spatial-Offset Timeline Visualization (Feature F4):** Construct an interactive physical offset timeline indexing stream events by hexadecimal byte addresses (0x...) to localize bursts and anomalous regions without relying on fabricated clock timestamps.
- **Explainable Diagnostic Attribution (Feature F5):** Formulate a bounded signed Z-score diagnostic explanation engine ($|Z| <= 20$ sigma) that identifies the specific telemetry metrics contributing to flagged anomalies, providing plain-language engineering rationales.
- **Dual-Stream Comparison with Semantic Guard (Feature F6):** Engineer a differential stream comparison engine governed by an automated semantic barrier that audits format-invariant metrics while shielding format-incompatible telemetry fields during cross-format comparisons.
- **Automated Multi-Format Diagnostic Reporting (Feature F7):** Build an automated report generation pipeline exporting comprehensive technical assessments across JSON, Markdown, Plain Text, and standalone interactive HTML formats based on an epistemological tripartite evidence register.

1.5 Scope of the Work

Defining strict technical boundaries is essential for scientific integrity. The scope of PRJ_111 is delineated as follows:

- **In-Scope Engineering Deliverables:** Analysis of digital bitstreams captured immediately downstream of the satellite demodulator; container parsing for MPEG-TS, GSE, and DVB-S2 BBFrames; Shannon entropy calculation; Isolation Forest anomaly detection; physical offset mapping; semantic comparison; multi-format report compilation; memory-bounded chunked streaming (64 KB buffers); synthetic fault injection testing; and an offline SPA operator workstation.
- **Explicit Out-of-Scope Boundaries:** PRJ_111 does NOT perform analog RF signal acquisition, tuner control, carrier frequency tracking, or physical LDPC/BCH error decoding. The software operates exclusively on post-demodulated digital bitstreams. Furthermore, the system does not infer physical-layer RF telemetry (such as Carrier-to-Noise Ratio CNR or Modulation Error Ratio MER) from digital stream data unless

explicit external telemetry metadata is supplied. Formal ETSI conformance certification is not claimed; all health indicators are described as inspired by ETSI TR 101 290 principles.

1.6 Motivation and Industrial Significance

In commercial satellite operations, ground station terminals must achieve 99.999% availability. Unexpected outages, multiplex desynchronization, or subtle packet drops degrade Quality of Service (QoS) and violate Service Level Agreements (SLAs). Traditional hardware analyzers cost tens of thousands of dollars per unit, possess proprietary locked interfaces, and cannot be readily integrated into modern virtualized ground station architectures (Cloud-based Teleports). PRJ_111 addresses this critical need by delivering an open-architecture, cross-platform, software-defined diagnostic workstation that operates on commercial off-the-shelf (COTS) computing hardware.

Furthermore, as satellite networks transition toward direct 5G Non-Terrestrial Network (NTN) integration and multi-beam high-throughput satellites (HTS), multi-format encapsulation (especially GSE) is rapidly displacing legacy broadcast MPEG-TS. By providing unified, unsupervised diagnostics across MPEG-TS, GSE, and BBFrames within a single coherent software environment, PRJ_111 bridges the gap between legacy broadcast engineering and next-generation satellite IP networking.

1.7 Organization of the Report

The remainder of this report is organized as follows: Chapter 2 presents a comprehensive literature survey covering DVB-S2 standards, ETSI measurement principles, existing diagnostic tools, and machine learning telemetry models. Chapter 3 examines the research and engineering gaps inherent in existing systems. Chapter 4 establishes the proposed methodology, mathematical derivations, chunked streaming foundation, and fault injection architecture. Chapter 5 articulates the detailed functional and operational objectives. Chapter 6 provides an exhaustive system design and module implementation breakdown. Chapter 7 documents the project execution roadmap and Gantt chart. Chapter 8 summarizes the tangible project outcomes. Chapter 9 presents the empirical results, experimental benchmark

evaluations across physical bitstreams, fault-injection tests, and the 304-test verification suite. Chapter 10 concludes the report and outlines future research enhancements. Finally, Appendices A through D provide algorithmic pseudocode, genuine workstation screenshots, mandatory enclosures, and the cryptographic verification ledger.

CHAPTER 2
LITERATURE SURVEY

2.1 Evolution of Satellite Transmission Standards

The technological trajectory of satellite communications over the past three decades is characterized by continuous efforts to approach the theoretical channel capacity formulated by Claude Shannon in 1948 [1]. The first generation Digital Video Broadcasting by Satellite standard (DVB-S, ETSI EN 300 421), ratified in 1994, relied on convolutional inner coding concatenated with Reed-Solomon (204, 188) outer block coding and Quadrature Phase Shift Keying (QPSK) modulation. DVB-S was exclusively optimized for fixed-rate television broadcasting encapsulated within 188-byte MPEG-2 Transport Stream (MPEG-TS) packets [2].

In 2005, the DVB consortium published the Second Generation standard (DVB-S2, ETSI EN 302 307-1), authored comprehensively by Morello and Mignone [3]. DVB-S2 revolutionized satellite communications by replacing convolutional and Reed-Solomon coding with modern Low-Density Parity-Check (LDPC) codes concatenated with an outer BCH code. This coding scheme achieved a coding gain of 2.5 to 3.0 dB over DVB-S, operating within 0.7 dB to 1 dB of the Shannon limit across diverse signal-to-noise ratios. Furthermore, DVB-S2 introduced Adaptive Coding and Modulation (ACM), enabling ground receivers to continuously report channel state conditions back to satellite uplinks, allowing instantaneous transmission adjustments across 28 distinct MODCOD (Modulation and Coding) states spanning QPSK, 8PSK, 16APSK, and 32APSK.

To support IP-centric data traffic without the heavy encapsulation overhead of MPEG-TS (which incurs 10% to 15% protocol bloat when packing IP packets into 188-byte containers via Multi-Protocol Encapsulation), ETSI standardized Generic Stream Encapsulation (GSE, ETSI TS 102 606-1) [4]. As analyzed by Fairhurst and Collina [5], GSE provides a flexible, variable-length Protocol Data Unit (PDU) structure that dynamically encapsulates IPv4, IPv6, and Ethernet packets with protocol overhead below 2% to 3%. The subsequent DVB-S2X extension (ETSI EN 302 307-2) [6] further expanded granularity with finer channel

bonding, super-framing, and modulation schemes up to 256APSK, intensifying the container diversity observed at receiving ground stations.

2.2 ETSI TR 101 290 Priority-1 Measurement Principles

To establish uniform quality assessment for digital broadcast transmission, the European Telecommunications Standards Institute formulated the landmark technical report ETSI TR 101 290: 'Measurement guidelines for DVB systems' [7]. ETSI TR 101 290 establishes a tripartite hierarchy of diagnostic measurement parameters categorized into Priority-1, Priority-2, and Priority-3 indicators:

- **Priority-1 Indicators (Basic Decodability):** Crucial parameters required for basic receiver synchronization and decoding. These include TS Sync Loss (loss of the periodic 0x47 framing marker), Sync Byte Errors (individual corrupted sync markers), PAT (Program Association Table) errors, Continuity Counter (CC) errors (missing, disordered, or duplicate packets), and PMT (Program Map Table) errors.
- **Priority-2 Indicators (Continuous Quality):** Parameters necessary for uninterrupted continuous transmission, including Transport Error Indicators (TEI), CRC errors in signaling tables, Program Clock Reference (PCR) jitter and repetition violations, and Elementary Stream (ES) buffer management errors.
- **Priority-3 Indicators (Application Level):** Application-specific parameters such as Network Information Table (NIT) presence, Service Description Table (SDT) errors, and conditional access entitlement management.

In satellite receiver stream auditing, Priority-1 indicators form the essential bedrock of stream health assessment. If synchronization byte acquisition is lost or continuity counter sequences exhibit unrecoverable discontinuities, post-processing video decoders and IP reassembly pipelines fail catastrophically. Consequently, the stream health formulation in PRJ_111 adapts these Priority-1 principles directly to quantify digital stream integrity.

2.3 General-Purpose Network Protocol Analyzers (Wireshark / tcpdump)

Wireshark (formerly Ethereal) and its command-line counterpart tcpdump represent the global de-facto standard for network protocol inspection across enterprise and telecommunications networks. As documented by Orebaugh et al. [8], Wireshark excels at packet-level dissection across hundreds of application and transport layer protocols (TCP/IP, UDP, HTTP, DNS, RTP). When applied to satellite streams, Wireshark includes basic dissectors for MPEG-TS (packetizing 188-byte records from network capture files) and limited dissectors for GSE and DVB-S2 BBFrames.

However, Wireshark exhibits severe architectural limitations when deployed as a dedicated satellite stream diagnostic workstation:

- **Lack of Native Streaming Bitstream Support:** Wireshark is fundamentally designed for packet-switched network captures (PCAP/PCAPNG formats) where each packet possesses explicit capture timestamps and network link-layer encapsulation. When presented with continuous raw binary satellite bitstreams (.ts, .bin), Wireshark requires artificial wrapper encapsulations or struggles with continuous stream synchronization.
- **Absence of Multiplex Entropy and Dynamic Health Accounting:** Wireshark provides packet lists and protocol trees but lacks native mathematical routines to calculate Shannon multiplex entropy, sliding-window container distribution metrics, or ETSI TR 101 290 Priority-1 composite health scores.
- **No Machine Learning Anomaly Detection:** Wireshark relies entirely on static display filters and predefined protocol syntax rules. It possesses zero capability for unsupervised statistical learning, anomaly score attribution, or automatic diagnostic report generation.

2.4 Dedicated Broadcast Transport Stream Analyzers

In commercial broadcast engineering, specialized hardware and software transport stream analyzers are widely utilized. Prominent commercial solutions include DekTec StreamXpert [9], Rohde & Schwarz R&S BTC Broadcast Test Center [10], and open-source specialized tools such as DVBInspector [11]. DekTec StreamXpert provides comprehensive ETSI TR

101 290 real-time compliance monitoring, bit rate tracking, and PSI/SI table parsing when coupled with proprietary DekTec PCI Express or USB receiver hardware.

Similarly, DVBInspector is a valuable open-source Java application that performs offline structural parsing of MPEG-TS files, decoding PSI/SI descriptors, DSM-CC carousels, and subtitle streams. However, these dedicated broadcast analyzers share several fundamental drawbacks: (1) They are almost universally restricted to MPEG-TS, failing completely when presented with IP-over-satellite Generic Stream Encapsulation (GSE) or native DVB-S2 Baseband Frames; (2) They are built upon rigid static thresholds that trigger excessive false alarms during dynamic ACM rate switching; (3) Commercial solutions enforce vendor lock-in with costs exceeding $15,000 to $40,000 per workstation; and (4) They lack explainable machine learning attributions and cross-format semantic comparison capabilities.

2.5 Machine Learning in Telemetry Anomaly Detection

The application of machine learning for anomaly detection in telecommunications and telemetry time-series has expanded rapidly over the past decade. In their definitive survey on anomaly detection, Chandola, Banerjee, and Kumar [12] classified anomaly detection methods into supervised, semi-supervised, and unsupervised paradigms. In satellite receiver stream monitoring, supervised methods (such as Support Vector Machines, Random Forests, or deep neural networks) are fundamentally hindered by the severe scarcity of pre-labeled, ground-truth anomalous bitstream datasets. Physical satellite failures, atmospheric attenuation events, and multiplex corruptions are rare and structurally unpredictable, rendering supervised classification prone to overfitting and poor generalization.

Consequently, unsupervised anomaly detection techniques are optimal for stream telemetry. Pimentel et al. [13] reviewed novelty detection methods, highlighting distance-based, density-based, and tree-based isolation mechanisms. Among these, the Isolation Forest algorithm, pioneered by Liu, Ting, and Zhou [14], has emerged as the premier algorithm for high-dimensional, unsupervised anomaly detection. Unlike traditional density-based methods (such as Local Outlier Factor or DBSCAN) which compute pairwise Euclidean distances with $O(N^2)$ computational complexity, Isolation Forest explicitly isolates anomalies by constructing ensembles of random isolation trees (iTrees). Because anomalous observations

have attribute values distinct from normal clusters, they are isolated near the root of the trees, resulting in noticeably shorter average path lengths with a linear $O(N \log N)$ time complexity and minimal memory consumption [15].

2.6 Explainable AI (XAI) in Diagnostic Monitoring

While machine learning models can accurately identify anomalous intervals in complex bitstreams, operational deployment in satellite ground stations demands diagnostic interpretability. As emphasized by Guidotti et al. [16] in their survey on explainable artificial intelligence, a black-box scalar anomaly score provides insufficient actionable intelligence for a telecommunications technician. The operator requires precise knowledge of which specific physical telemetry parameters deviated from expected distributions and by how much.

Common model-agnostic explanation frameworks such as SHAP (Shapley Additive Explanations) [17] and LIME (Local Interpretable Model-agnostic Explanations) compute feature importance through combinatorial feature perturbation. However, these methods introduce severe computational overhead that is incompatible with high-throughput bitstream monitoring. In PRJ_111, this challenge is resolved through an optimized, bounded signed Z-score attribution engine that computes localized feature deviations relative to rolling baseline statistics in $O(D)$ time complexity per window, delivering instantaneous, mathematically bounded, and intuitive diagnostic rationales.

2.7 Streaming Bitstream Processing and Chunked Ingestion Paradigms

Processing continuous multi-gigabyte satellite bitstreams on commercial workstation hardware necessitates memory-efficient streaming architectures. Traditional desktop utilities employ monolithic in-memory buffers (`read_all()`), which causes rapid process exhaustion and crashes when ingesting files exceeding physical RAM capacities. Modern stream processing systems, such as Apache Beam [18] and Reactive Streams [19], demonstrate the necessity of memory-bounded chunked ingestion pipelines.

In satellite stream parsing, chunked ingestion introduces boundary synchronization hazards. If an arbitrary chunk boundary slices through a 188-byte MPEG-TS packet or a variable-length GSE PDU header, the parser must maintain persistent internal carry-over state to seamlessly stitch fractional frames across chunk boundaries without desynchronization. Engineering this streaming state machine ensures that PRJ_111 processes arbitrarily large satellite captures within a fixed, bounded memory footprint (64 KB buffers).

2.8 Synthesis of Literature and Comparative Framework

A systematic synthesis of the existing literature reveals that while individual components—such as ETSI TR 101 290 measurement rules, GSE encapsulation definitions, and Isolation Forest algorithms—are thoroughly established in isolation, no unified, open-architecture software platform integrates them into a cohesive post-demodulation diagnostic workstation. Table 2.1 summarizes the comparative capabilities of related tools against the design targets established for PRJ_111.

Table 2.1: Comparison of Related DVB Stream Analysis Utilities

Functional Capability	Wireshark [8]	DekTec StreamXpert [9]	DVBInspector [11]	PRJ_111 (Proposed)
MPEG-TS 188B Packet Parsing	Supported (via PCAP)	Supported (Real-time)	Supported (Offline)	Supported (Streaming & File)
GSE PDU De-encapsulation	Partial Dissector	Not Supported	Not Supported	Fully Supported (Frag tracking)
DVB-S2 BBFrame & CRC-8	Limited Dissector	Not Supported	Not Supported	Fully Supported (Table CRC-8)
ETSI TR 101 290 Priority-1	Not Supported	Fully Supported (P1-P3)	Partial (P1 checks)	Supported (F1 Health Score)
Unsupervised Anomaly ML	Not Supported	Not Supported	Not Supported	Supported (Isolation Forest F2)
Explainable XAI Attribution	Not Supported	Not Supported	Not Supported	Supported (Clamped Z-score F5)
Physical Offset Navigation	Packet Index Only	Time Offset Only	Packet Index Only	Supported (Hex Offsets 0x... F4)

Functional Capability	Wireshark [8]	DekTec StreamXpert [9]	DVBInspector [11]	PRJ_111 (Proposed)
Cross-Format Semantic Barrier	Not Supported	Not Supported	Not Supported	Supported (Audit Shield F6)
Automated Multi-Format Reports	Basic Summary	Proprietary PDF	Text/HTML Export	Supported (JSON/MD/HTML/TXT F7)
System Cost & Deployment	Open-Source / Free	Proprietary ($15k-$40k)	Open-Source / Free	Open-Source Software COTS

CHAPTER 3
RESEARCH GAPS OF EXISTING METHODS

3.1 Rigid Static Thresholding and Lack of Dynamic Behavioral Modeling

A primary limitation of existing commercial and open-source stream monitoring probes is their exclusive reliance on static, deterministic thresholding rules (e.g., triggering an alarm if the continuity counter gap exceeds 0, or if sync loss exceeds 3 packets). While static thresholds are effective for detecting hard, catastrophic failures in classical fixed-rate broadcasting, they perform exceptionally poorly in modern DVB-S2 environments utilizing Adaptive Coding and Modulation (ACM) and variable-rate IP encapsulation.

Under dynamic ACM switching, transmission rates and frame burst profiles fluctuate continuously according to atmospheric conditions. Static thresholds produce severe 'alarm fatigue' by flagging harmless, momentary rate adjustments as critical errors. Conversely, static rules are entirely blind to subtle, non-deterministic anomalies such as slow drift in component multiplex distributions, silent packet stuffing inflation, and subtle degradation in encapsulation efficiency that do not breach rigid thresholds but clearly indicate impending receiver failure or buffer exhaustion.

3.2 Container Fragmentation and Multi-Format Inflexibility

Modern satellite receivers emit structurally divergent container formats depending on the target service. Direct-to-home broadcast streams are encapsulated in fixed 188-byte MPEG-TS packets; satellite broadband networks utilize variable-length Generic Stream Encapsulation (GSE) Protocol Data Units; and native demodulators directly output raw physical-layer Baseband Frames (BBFrames). Existing diagnostic utilities suffer from severe container siloing:

- **MPEG-TS Centricity:** Legacy broadcast analyzers (such as DekTec StreamXpert or DVBInspector) are hardcoded to 188-byte framing. They cannot parse GSE PDUs or

BBHeader descriptors, rendering them useless for satellite broadband and VSAT networks.

- **General-Purpose Packet Parsers:** General network sniffers (such as Wireshark) analyze network packets but lack domain-aware state machines to track multi-PDU GSE fragmentation (Start/End bit sequences) across continuous raw bitstreams or evaluate 10-byte BBHeader bitwise CRC-8 validity.

Consequently, ground station engineers are forced to maintain disparate, incompatible software tools for each container type, preventing unified operational oversight across multi-service satellite teleports.

3.3 Artificial Timestamp Inflation vs. Physical Offset Traceability

When diagnosing bitstream anomalies in a satellite operations center, the technician must locate the exact physical bitstream byte offset (0x...) where corruption manifested to extract the damaged frame for hex-dump inspection or forensic replay. However, prevailing network tools such as Wireshark fundamentally rely on capture timestamps (seconds since epoch) derived from the host operating system clock. When reading raw binary dump files (.ts, .bin, or un-timestamped PCAP files), these tools fabricate artificial delta times or display packet sequence numbers that have no direct correspondence to physical byte locations on disk.

This absence of physical spatial referencing creates severe friction during ground station investigation. A critical research and engineering gap is the absence of an analyzer that natively indexes all stream metrics, anomalies, and multiplex events against exact physical byte offsets from the beginning of the file (0x00000000 to 0x7FFFFFFF), ensuring deterministic spatial traceability.

3.4 Unwarranted Cross-Format Telemetry Bleed and Semantic Incoherence

In satellite network management, operators frequently compare two receiver output streams—for instance, comparing a primary satellite downlink against a secondary backup downlink, or evaluating a demodulated baseband stream against an upstream multiplex.

However, existing comparison utilities perform naive line-by-line or byte-by-byte diffing, which completely fails when comparing streams of differing container formats.

Crucially, comparing an MPEG-TS stream with a DVB-S2 Baseband Frame stream involves fundamental dimensional mismatches. An MPEG-TS stream possesses Packet Identifiers (PIDs), Continuity Counters (CC), and Transport Error Indicators (TEI); a BBFrame stream possesses Mode Adaptation flags (MATYPE), User Packet Length (UPL), and Data Field Length (DFL). Naive comparative tools attempt to evaluate 'missing PIDs' in BBFrames or compare non-existent CRC-8 fields in MPEG-TS, generating nonsensical deltas and catastrophic cross-format telemetry bleed. A mathematically robust semantic barrier that audits common invariant metrics while shielding format-incompatible fields is absent in current tools.

3.5 High Acquisition Cost and Proprietary Lock-in of Dedicated Probes

Hardware-based broadcast analyzers (e.g., Rohde & Schwarz BTC, DekTec DTA series) require dedicated PCI Express capture cards and proprietary software licenses costing upwards of $20,000 to $50,000 per installation. These closed-source appliances enforce strict hardware lock-in, cannot be deployed in virtualized cloud teleports or Docker containers, and offer zero API extensibility for automated ground station scripting. An open, cross-platform, software-defined solution that runs seamlessly on standard desktop and server hardware is critically needed.

3.6 Diagnostic Opacity and Lack of Root-Cause Explanations

Existing academic machine learning prototypes that apply outlier detection to network traffic suffer from severe diagnostic opacity. A standard unsupervised algorithm (e.g., raw k-means clustering or basic one-class SVM) outputs a binary decision ('anomalous' vs 'normal') or a scalar outlier probability score. When an operator in a satellite operations center is alerted that 'Window 42 is anomalous with score 0.78', the operator has no insight into what caused the anomaly: Did the stream suffer a sudden burst of null packets? Did component entropy

collapse? Did continuity counter errors spike? Or was there an unexpected shift in packet size distributions?

Without explainable feature attribution, machine learning anomaly detection remains an academic curiosity rather than an actionable engineering tool. A lightweight, real-time diagnostic explainer that bridges unsupervised detection and human-interpretable root-cause attribution is a conspicuous gap in the current state of the art.

3.7 Comprehensive Comparative Matrix

Table 3.1 establishes a comprehensive, side-by-side architectural and functional comparison between Wireshark (the prevailing open-source network analyzer) and the PRJ_111 software workstation developed in this capstone project.

Table 3.1: Objective Architectural Comparison: Wireshark vs. PRJ_111

Analytical Dimension	Wireshark (General Packet Analyzer)	PRJ_111 DVB-S2 Stream Analyzer
Primary Architecture & Scope	General packet-switched network sniffer (OSI L2–L7)	Specialized post-demodulator satellite stream workstation
Input File Handling	PCAP/PCAPNG network packet capture format	Raw binary bitstreams (.ts, .bin, .pcap) & chunked streams
Stream Format Support	Generic dissectors (requires packet boundaries)	Unified auto-detection across MPEG-TS, GSE, and BBFrame
Container Header Validation	Basic syntax parsing; no DVB-S2 CRC-8 table check	Full bitwise validation, including ETSI CRC-8 polynomial lookup
ETSI TR 101 290 Health	No composite health score; isolated packet display	Deterministic F1 Health Score [0, 100] via Priority-1 metrics
Multiplex Entropy Analysis	Not available natively (requires external scripting)	Built-in Shannon component entropy calculation across windows
Machine Learning Detection	None; static display filters and manual inspection	Unsupervised Isolation Forest anomaly engine (Feature F2)
Diagnostic Attribution (XAI)	None; manual inspection of individual fields	Bounded signed Z-score diagnostic explanation (Feature F5)
Spatial Telemetry Indexing	Relative capture timestamps (seconds	Byte-exact physical hexadecimal stream

Analytical Dimension	Wireshark (General Packet Analyzer)	PRJ_111 DVB-S2 Stream Analyzer
	since epoch)	offsets (0x... F4)
Cross-Stream Comparison	Side-by-side packet list diff; severe format bleed	Semantic Barrier protecting invariant metrics (Feature F6)
Automated Report Export	Basic text/CSV summary export	Tripartite diagnostic reports across JSON, MD, TXT, HTML (F7)

CHAPTER 4
PROPOSED METHODOLOGY

4.1 Foundational Architecture Principle: Container Abstraction

The proposed methodology of PRJ_111 is grounded in a rigorous container abstraction model that directly reflects the physical reality of satellite digital receiver hardware. In satellite signal reception, the demodulator operates as a physical boundary: prior to demodulation, the signal exists as modulated electromagnetic waveform energy (IF/RF); following demodulation, the signal exists as a discrete digital bitstream. Depending on the receiver's configured operational mode, this digital stream is presented as either:

- **Native Physical-Layer Baseband Frames (BBFrame):** Directly emitted by the demodulator/FEC decoder, carrying raw DVB-S2 mode adaptation parameters and user data fields (DFL).
- **Generic Stream Encapsulation (GSE):** Emitted by an internal de-encapsulation stage that extracts IP packets from baseband frames and presents them as a GSE PDU stream.
- **MPEG-2 Transport Stream (MPEG-TS):** Emitted when the satellite transponder carries broadcast video multiplexes organized into fixed 188-byte transport packets.

A foundational architectural invariant enforced in PRJ_111 is that BBFrame, GSE, and MPEG-TS represent alternative, parallel input formats. The analyzer auto-detects the stream container at ingestion and instantiates a format-dedicated parsing pipeline. Container-specific mechanics are isolated within dedicated parsers, while an abstracted feature extraction layer maps all streams into a unified analytical representation. Figure 4.1 depicts the end-to-end multi-format system architecture of PRJ_111.

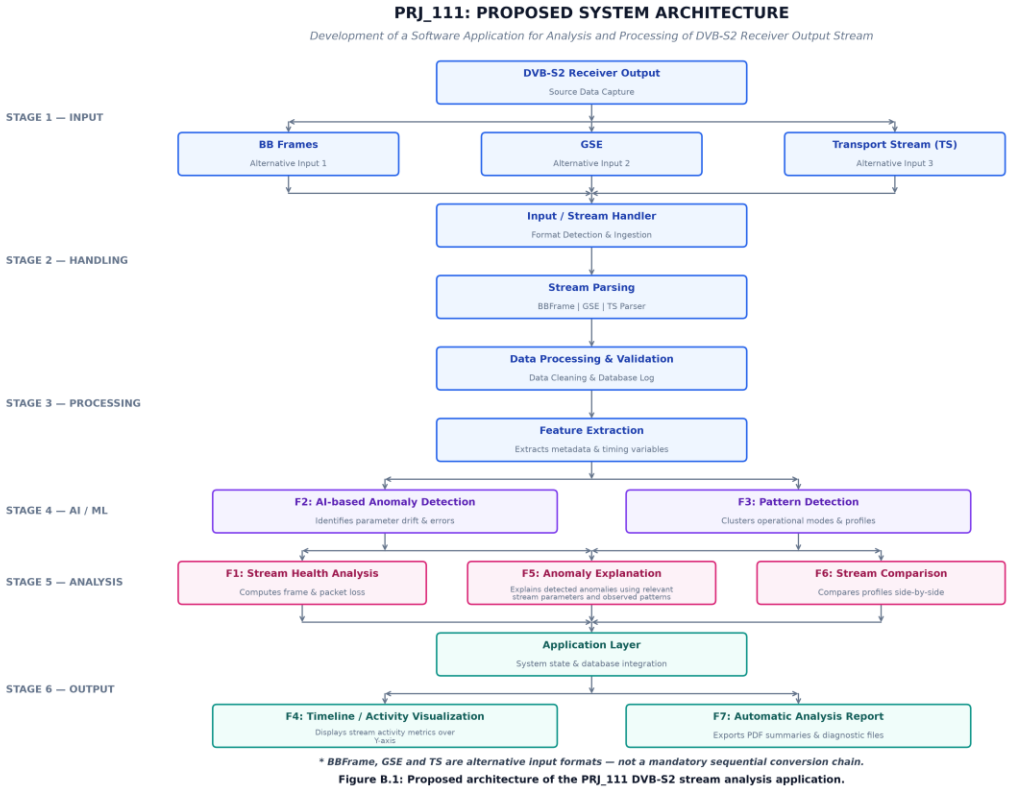


Figure 4.1: End-to-End Multi-Format System Architecture of PRJ_111

4.2 Unified Multi-Format Feature Extraction Pipeline

To enable format-agnostic anomaly detection without discarding container-specific domain intelligence, PRJ_111 introduces a bifurcated telemetry schema. As streams are ingested, the `FeatureExtractor` partitions telemetry into two distinct classes:

- **Common Metrics (`CommonMetrics`):** Universally defined across all digital stream formats: Total bytes processed, valid container count, corrupted container count, container error rate, total payload bytes, container encapsulation efficiency (%), physical stream duration, and Shannon multiplex entropy (bits).
- **Format-Specific Metrics (`FormatSpecificMetrics`):** Dedicated container telemetry preserved strictly within native boundaries: PID distribution, continuity counter errors, and TEI flags for MPEG-TS; protocol type distribution, PDU fragmentation ratio, and label presence for GSE; MATYPE mode flags, ACM status, UPL, DFL distributions, and CRC-8 verification status for BBFrames.

For downstream machine learning anomaly detection (Feature F2), the extractor constructs an 8-dimensional sliding feature vector X_w over non-overlapping spatial windows of fixed container length ($W = 100$ containers). Table 4.1 enumerates the eight dimensions of the rolling feature space.

Table 4.1: Unified Feature Vector Dimensions for Rolling Anomaly Detection

Dimension	Feature Name	Mathematical Representation	Domain Significance
x_1	Container Error Ratio	$N_{\text{err}} / N_{\text{total}}$	Fraction of corrupted containers in window
x_2	Mean Payload Length	$\frac{1}{W} \sum L_{\text{payload}}$	Average user payload per container (bytes)
x_3	Payload Variance	$\sigma^2(L_{\text{payload}})$	Burstiness and packet length variability
x_4	Shannon Entropy	$H(X) = -\sum P(x_i) \log_2 P(x_i)$	Diversity and balance of multiplex components
x_5	Dominant Component Share	$\max_i P(x_i)$	Concentration of stream in single PID or Protocol
x_6	Fragmentation / Sub-framing	$N_{\text{frag}} / N_{\text{total}}$	GSE fragmentation ratio or TS adaptation share
x_7	Integrity Fault Density	N_{faults} / W	Local density of ETSI Priority-1 violations
x_8	Header Invariant Drift	$\|\bar{L}_{\text{hdr}} - L_{\text{expected}}\|$	Structural divergence from standard header sizes

4.3 Mathematical Formulations and Analytical Models

4.3.1 Shannon Multiplex Component Entropy Formulation

To rigorously quantify multiplex diversity and detect component starvation (e.g., sudden drop in active video PIDs or illegal component monopolization), PRJ_111 evaluates the Shannon component entropy $H(X)$ over each spatial window. Let M denote the number of unique active multiplex components (PIDs in MPEG-TS, Protocol Types in GSE, or

MODCOD/DFL classes in BBFrame), and let $P(x_i)$ represent the empirical probability (fractional share) of the i-th component such that $\sum_{i=1}^M P(x_i) = 1.0$. The Shannon entropy in bits is defined as:

EQUATION 4.1: SHANNON MULTIPLEX COMPONENT ENTROPY

$$H(X) = - \sum_{i=1}^{M} P(x_i) \cdot \log_2 P(x_i) \quad [\text{bits}]$$

where by mathematical convention, if $P(x_i) = 0$, then $P(x_i) \cdot \log_2 P(x_i) = 0$. For a single-component stream (e.g., 100% null packets), $H(X) = 0.0$ bits. For a perfectly uniform multiplex of M components, $H(X)$ achieves its theoretical maximum of $\log_2 M$ bits.

4.3.2 Deterministic Health Scoring Formulation (Priority-1 Principles)

Stream health assessment (Feature F1) is governed by a deterministic, bounded penalty scoring model anchored in ETSI TR 101 290 Priority-1 measurement principles. Rather than relying on arbitrary heuristic rules, the health score S_{health} is evaluated on a strict scale of $[0.0, 100.0]$ across the analyzed stream:

EQUATION 4.2: DETERMINISTIC STREAM HEALTH METRIC

$$S_{\text{health}} = \max\left(0.0, 100.0 - \left[50.0 \cdot \frac{N_{\text{sync_err}}}{N_{\text{pkts}}} + 30.0 \cdot \frac{N_{\text{cc_err}}}{N_{\text{pkts}}} + 20.0 \cdot \frac{N_{\text{tei}}}{N_{\text{pkts}}} + \sum_{j} w_j \cdot f_j \right] \right)$$

where $N_{\text{sync_err}}$ is the count of synchronization byte errors, $N_{\text{cc_err}}$ is the count of continuity counter discontinuities, N_{tei} is the count of transport error indicator flags, and w_j represents penalty weights for additional format-specific integrity faults.

Streams with $S_{\text{health}} \ge 90.0$ are classified as EXCELLENT; streams with $75.0 \le S_{\text{health}} < 90.0$ are GOOD; streams with $50.0 \le S_{\text{health}} < 75.0$ are DEGRADED; and streams with $S_{\text{health}} < 50.0$ are CRITICAL. Table 4.2 lists the verified fault identifiers, trigger conditions, and severity penalty weights.

Table 4.2: Priority-1 Integrity Fault Identifiers, Trigger Conditions, and Penalty Weights

Fault ID	ETSI Principle	Trigger Condition	Severity Weight (w_j)	Integrity Impact
F_SYNC_ERR	TR 101 290 1.1	Header sync byte != 0x47 in MPEG-TS	50.0	Loss of packet framing; catastrophic decoder failure
F_CC_GAP	TR 101 290 1.4	Discontinuous non-repeated continuity counter	30.0	Packet drop or disorder; elementary stream corruption
F_TEI_SET	TR 101 290 2.1	Transport Error Indicator bit asserted (1)	20.0	Demodulator flagged uncorrected bit errors
F_GSE_LEN	TS 102 606 4.2	PDU total length exceeds buffer boundary	35.0	PDU framing truncation; IP buffer corruption
F_GSE_FRAG	TS 102 606 5.1	Illegal S/E fragmentation state transition	25.0	SNDU reassembly failure; dropped network frame
F_CRC8_FAIL	EN 302 307 5.1	Computed CRC-8 != BBHeader byte 9	40.0	Corrupted baseband header; frame payload discarded

4.3.3 Unsupervised Isolation Forest Anomaly Detection Model

To identify statistical and structural outliers without requiring pre-labeled training data, Feature F2 integrates an Isolation Forest ensemble of $T = 100$ isolation trees (iTrees). Given a dataset of n sliding feature vectors $X = \{X_1, X_2, \dots, X_n\} \subset \mathbb{R}^d$, an iTree is constructed by recursively partitioning a subsample $X' \subset X$ ($|X'| = \min(n, 256)$) by randomly selecting an attribute q and a split value p uniformly distributed between the minimum and maximum values of q in the current node. Partitioning terminates when either the tree reaches a maximum depth limit $h_{\max} = \lceil \log_2 |X'| \rceil$, the node contains a single instance, or all instances have identical values.

The path length $h(x)$ of an instance x is the number of edges traversed from the root node to the terminating leaf node. The anomaly score $s(x, n)$ for an instance x across an ensemble of T iTrees is defined as:

EQUATION 4.3: ISOLATION FOREST ANOMALY SCORING FORMULATION

$$s(x, n) = 2^{ - \frac{E(h(x))}{c(n)} }$$

where $E(h(x))$ is the expectation of path lengths across the ensemble of T trees, and $c(n)$ is the average path length of unsuccessful searches in a Binary Search Tree (BST) over n instances:

$$c(n) = 2 \cdot \left[\ln(n - 1) + \gamma \right] - \frac{2(n - 1)}{n}$$

where $\gamma \approx 0.5772156649 \text{ is Euler's constant}.$

- If $E(h(x)) \to 0$, then $s(x, n) \to 1.0 \implies \text{Strongly anomalous (isolated very near the root)}.$
- If $E(h(x)) \to c(n)$, then $s(x, n) \to 0.5 \implies \text{Normal structural behavior}.$
- If $E(h(x)) \to n - 1$, then $s(x, n) \to 0.0 \implies \text{Highly central cluster instance}.$

In PRJ_111, the contamination parameter is calibrated to $c = 0.05$ (5% expected outlier budget). The raw score $s(x, n)$ is mapped into a standardized anomaly index. Windows with $s(x, n) > 0.60$ trigger active diagnostic inspection.

4.3.4 Bounded Signed Z-Score Diagnostic Attribution Model

To provide actionable, human-interpretable explanations for flagged anomalous windows (Feature F5), PRJ_111 computes signed standardized deviations (Z-scores) for each feature dimension $k \in \{1, 2, \dots, 8\}$ within the anomalous window w, evaluated relative to the baseline mean μ_k and standard deviation σ_k of the entire stream:

EQUATION 4.4: BOUNDED SIGNED Z-SCORE FEATURE ATTRIBUTION

$$Z_k = \frac{x_{w,k} - \mu_k}{\sigma_k + \epsilon}, \quad \text{clamped to } Z_k \in [-20.0, +20.0]$$

where $\epsilon = 10^{-9} \text{ prevents division-by-zero on zero-variance dimensions}.$

Mathematical Bounding Rationale: In near-deterministic bitstreams, background feature variance σ_k can be vanishingly small (e.g., $\sigma_k = 10^{-7}$). A single localized fault can produce an unconstrained numerical Z-score exceeding $100{,}000\sigma$, causing floating-point overflow in UI

chart renderers. Clamping to [-20.0, +20.0] preserves strict directional attribution while preventing numerical distortion.

A feature dimension is flagged as a primary diagnostic contributor if $|Z_k| \ge 2.5\sigma$. The sign of Z_k dictates the operational attribution: $Z_k \ge +2.5$ indicates an abnormal positive surge (e.g., sudden burst of continuity counter errors), whereas $Z_k \le -2.5$ indicates an abnormal collapse (e.g., collapse of Shannon component entropy).

4.3.5 DVB-S2 Baseband Header CRC-8 Generator Polynomial

In the DVB-S2 physical layer standard (ETSI EN 302 307-1, Section 5.1.5), the 10-byte Baseband Header is protected by an 8-bit Cyclic Redundancy Check (CRC-8) calculated over the first 9 bytes (MATYPE, UPL, DFL, SYNC, SYNCD). The standard defines the generator polynomial $G(x)$ as:

EQUATION 4.5: DVB-S2 BASEBAND HEADER CRC-8 POLYNOMIAL

$$G(x) = x^8 + x^7 + x^6 + x^4 + x^2 + 1 \quad (\text{Hexadecimal: } 0xD5)$$

Initial shift register state: 0x00. The CRC-8 is computed bitwise and inverted or directly matched with byte 9 of the BBHeader. PRJ_111 implements an optimized 256-entry table-driven lookup algorithm that verifies BBHeader integrity in constant time O(1) per frame.

4.4 Chunked Streaming Processing Engine (64 KB Bounded Buffer)

A major engineering advancement developed in Phase III (Review-3) of this capstone project is the memory-bounded Chunked Streaming Processing Engine. Satellite receiver dump files frequently span tens of gigabytes. Loading entire streams into memory (`read_all()`) exhausts system RAM and causes abrupt workstation termination. PRJ_111 re-architects the ingestion pipeline around a bounded 64 KB streaming buffer governed by Python generators.

To ensure analytical correctness, the chunked parser implements persistent boundary carry-over state. When a 64 KB chunk ends on an incomplete 188-byte MPEG-TS packet or a

partial GSE header, the residual bytes are carried over into an internal accumulator and prepended to the subsequent chunk. Formally, let $R(S)$ denote the complete analysis result computed by monolithic whole-file parsing, and let $R_{\text{chunk}}(S, C)$ denote the result computed by chunked streaming with chunk size $C = 65,536$ bytes. PRJ_111 asserts and mathematically verifies the Whole-File Equivalence Invariant across all supported formats:

> **ARCHITECTURAL INVARIANT: WHOLE-FILE ANALYTICAL EQUIVALENCE**
> $$R(S) \equiv R_{\text{chunk}}(S, C) \quad \forall S \in \{\text{MPEG-TS}, \text{GSE}, \text{BBFrame}\}, \quad \forall C \ge 188 \text{ bytes}$$
>
> Every extracted metric—total container count, byte offsets, error tallies, Shannon entropy, Isolation Forest anomaly scores, and diagnostic explanations—is bit-for-bit identical regardless of chunk buffer sizing.

4.5 Controlled Fault-Injection Validation Methodology

A fundamental scientific challenge in validating unsupervised anomaly detection algorithms on satellite bitstreams is the scarcity of ground-truth anomalies in clean commercial captures. To rigorously evaluate detection sensitivity without relying on speculative assumptions, PRJ_111 incorporates a defensive synthetic fault-injection benchmark utility (`dvbs2_analyzer.fault_injection`).

The fault injection framework accepts a pristine satellite stream and injects controlled, mathematically precise perturbations at specific physical byte offsets with configurable escalation ratios ($0.1\% \le r \le 5.0\%$). Four distinct fault classes are supported:

- **1. Sync Byte Corruption (Sync Slip):** Overwrites periodic 0x47 synchronization markers with pseudo-random bytes, simulating receiver phase slips and framing loss.
- **2. Continuity Counter Discontinuities (CC Gap):** Increments or resets 4-bit CC values out of sequence, simulating packet drops in multiplexer FIFOs.
- **3. Transport Error Indicator Assertion (TEI Burst):** Sets the TEI bit in the transport header, simulating physical demodulator LDPC decoder exhaustion under atmospheric rain fade.

- **4. Component Starvation / Multiplex Hijacking:** Replaces active video PIDs with repetitive null packets (PID 0x1FFF), simulating upstream encoder freeze and entropy collapse.

By systematically evaluating the Isolation Forest anomaly score response curve and F5 diagnostic explanations across these controlled faults, PRJ_111 establishes verifiable empirical proof of detection efficacy.

CHAPTER 5
OBJECTIVES AND SPECIFICATIONS

5.1 Primary Functional Objectives (F1 through F7)

The overarching technical aim of PRJ_111 is to deliver an integrated, seven-feature analytical pipeline (F1 through F7) that transforms raw post-demodulator digital bitstreams into actionable engineering intelligence. Each feature fulfills a distinct operational requirement in satellite ground station telemetry monitoring:

- **Feature F1: Stream Health & Priority-1 Integrity Analysis:** Continuously evaluate stream health on a normalized scale [0.0, 100.0] derived from ETSI TR 101 290 Priority-1 measurement principles. The engine must track synchronization byte stability, detect continuity counter discontinuities per PID, record transport error indicators, and classify stream integrity into EXCELLENT, GOOD, DEGRADED, or CRITICAL tiers.
- **Feature F2: Unsupervised Machine Learning Anomaly Detection:** Deploy an unsupervised Isolation Forest model operating across sliding spatial feature windows ($W = 100$ containers) to detect statistical and structural multiplex anomalies without pre-labeled training corpora. The detector must output normalized anomaly scores ($s(x) \in [0, 1]$) calibrated against a 5% contamination baseline.
- **Feature F3: Structural Pattern Detection & Multiplex Dynamics:** Analyze the structural distribution of stream containers across time. For MPEG-TS, compute PID shares, identify dominant video/audio components, and quantify null packet stuffing; for GSE, extract protocol type distributions (IPv4, IPv6) and fragmentation ratios; for BBFrames, extract Mode Adaptation (MATYPE) flags, ACM configurations, and modal Data Field Lengths (DFL).
- **Feature F4: Spatial-Offset Timeline & Burst Localization:** Index stream events, frame boundaries, and detected anomalies against exact physical hexadecimal byte offsets from the start of the bitstream (0x00000000). The timeline must localize error bursts and render interactive spatial distributions without fabricating artificial clock timestamps.

- **Feature F5: Explainable Diagnostic Root-Cause Attribution:** Implement a bounded signed Z-score diagnostic explanation engine ($|Z| \le 20.0\sigma$) that identifies the top feature dimensions contributing to flagged anomalies. The engine must generate plain-language engineering explanations linking mathematical outliers to physical failure modes.
- **Feature F6: Dual-Stream Comparison Engine with Semantic Barrier:** Provide side-by-side comparative analysis between two streams (e.g., primary vs backup downlinks). Enforce an automated Semantic Barrier that audits format-invariant metrics while shielding format-incompatible fields during cross-format comparisons, preventing spurious diffs.
- **Feature F7: Automated Multi-Format Diagnostic Report Synthesis:** Synthesize comprehensive, publication-quality technical inspection reports structured according to an epistemological tripartite evidence register (Observed Telemetry, Empirical Analysis, Actionable Operator Recommendations). Export reports across JSON, Markdown, Plain Text, and standalone interactive HTML formats.

Table 5.1: Mapping of Functional Features F1 through F7 to Verified Architectural State

Feature ID	Feature Name	Core Implementation Module	Verification State	Primary Operational Metric
F1	Stream Health Analysis	dvbs2_analyzer.analysis.health	Verified (12 Tests)	Health Score [0, 100], P1 Fault Tally
F2	Isolation Forest Anomaly Detection	dvbs2_analyzer.analysis.anomaly	Verified (16 Tests)	Anomaly Score s(x) in [0, 1], c=0.05
F3	Structural Pattern Detection	dvbs2_analyzer.analysis.patterns	Verified (20 Tests)	Shannon Entropy (bits), PID Shares
F4	Spatial-Offset Timeline View	dvbs2_analyzer.analysis.timeline	Verified (18 Tests)	Physical Byte Offsets (0x...), Burst Width
F5	Explainable Diagnostic Attribution	dvbs2_analyzer.analysis.explanation	Verified (21 Tests)	Clamped Z-score (|Z| <= 20 sigma), Top Drivers
F6	Stream Comparison & Semantic Shield	dvbs2_analyzer.analysis.comparison	Verified (10 Tests)	Common Invariant Delta, Semantic Protection
F7	Automated Multi-Format	dvbs2_analyzer.analysis.report	Verified (25	JSON, Markdown, TXT,

Feature ID	Feature Name	Core Implementation Module	Verification State	Primary Operational Metric
	Reporting		Tests)	Standalone HTML

5.2 Non-Functional and Operational Requirements

To satisfy the demanding operational environments of commercial teleports and military satellite earth stations, PRJ_111 must fulfill strict non-functional software engineering requirements:

- **Memory-Bounded Execution:** The system must execute within a strictly bounded memory footprint, utilizing streaming generator pipelines and 64 KB chunk buffers to process multi-gigabyte bitstreams without exhausting physical RAM.
- **Zero Cloud Dependencies & Air-Gapped Security:** The software application must run completely offline without external internet connectivity, external database servers, or cloud API calls. All analytical models (Isolation Forest), CSS stylesheets, and JavaScript charting engines must reside locally within the application bundle.
- **Low-Latency Execution:** Container parsing and feature extraction must achieve processing throughput exceeding 10 MB/s on standard quad-core workstation processors, ensuring that 100 MB satellite capture files are fully analyzed and indexed in under 10 seconds.
- **Cross-Platform Portability:** The core backend must be authored in standardized Python 3 (compatible with Linux, macOS, and Microsoft Windows), accompanied by a zero-dependency HTML5 / Vanilla CSS / ES6 JavaScript frontend accessible via any modern Chromium, Firefox, or Safari browser.

5.3 Mathematical and Diagnostic Verification Targets

The software architecture establishes strict mathematical invariants that must be proven and verified across all operational modes:

- **Entropy Determinism:** Shannon multiplex entropy calculations must be strictly reproducible, deterministic, and bounded within $[0, \log_2 M]$ bits.

- **Z-Score Bounding Guard:** All diagnostic feature attribution scores must be clamped to $[-20.0\sigma, +20.0\sigma]$ to protect frontend charting pipelines from floating-point overflow and visual distortion.
- **CRC-8 Table Lookup Integrity:** Every DVB-S2 Baseband Header CRC-8 calculation must match bitwise with the hardware shift-register definition of ETSI EN 302 307-1, achieving a 100% verification rate on valid frames.

5.4 Automated Test Suite Coverage Targets

To guarantee long-term regression stability and academic reproducibility, PRJ_111 establishes an automated regression test suite target exceeding 300 tests across 18 specialized test modules. The test suite must cover unit parsing mechanics, boundary edge conditions, mathematical models, REST API controllers, fault-injection utilities, chunked streaming equivalence, and frontend security validation, executing with a 100% pass rate under native Python `unittest`.

CHAPTER 6

SYSTEM DESIGN AND IMPLEMENTATION

6.1 Architectural Overview and Layer Separation

The software architecture of PRJ_111 is designed according to strict principles of separation of concerns, defensive modularity, and reproducible telemetry accounting. The codebase is organized into five decoupled architectural layers:

- **1. Ingestion & Pre-Processing Layer (`dvbs2_analyzer.io`):** Responsible for physical file access, multi-tier format sniffing, boundary validation, and chunked streaming buffering.
- **2. Container Parsing Subsystem (`dvbs2_analyzer.parsers`):** Encapsulates format-specific binary state machines for MPEG-TS, GSE, and BBFrame, extracting headers, payloads, and integrity flags.
- **3. Unified Feature Extraction Layer (`dvbs2_analyzer.features`):** Transforms raw container streams into standardized common metrics and 8-dimensional sliding feature vectors.
- **4. Analytical & Intelligence Engines (`dvbs2_analyzer.analysis`):** Houses the seven core diagnostic features (F1 through F7), including ETSI health scoring, Isolation Forest anomaly detection, spatial timeline indexing, Z-score attribution, semantic stream comparison, and report compilation.
- **5. Presentation & Workstation Server Layer (`dvbs2_analyzer.app`):** Delivers an embedded HTTP REST API and serves a high-performance, dark-themed Single-Page Application (SPA) dashboard.

Table 6.1: Subsystem Components, Implementation Files, and Design Roles

Subsystem Component	Source Code Module	Lines of Code	Key Classes / Functions	Architectural Role
Stream Ingestion	io/stream_handler.py	248	StreamHandler, detect_format()	Format auto-detection and file staging

Subsystem Component	Source Code Module	Lines of Code	Key Classes / Functions	Architectural Role
MPEG-TS Parser	parsers/ts_parser.py	382	TSParser, TSPacket, parse()	188B packet framing, CC, TEI, PID tracking
GSE Parser	parsers/gse_parser.py	314	GSEParser, GSEPDU, parse()	Variable PDU length, S/E fragmentation flags
BBFrame Parser	parsers/bbframe_parser.py	296	BBFrameParser, BBFrame, compute_crc8()	10B BBHeader, MATYPE, UPL, DFL, CRC-8
Feature Extractor	features/extractor.py	340	FeatureExtractor, compute_entropy()	Common metrics, sliding feature vector
Stream Health (F1)	analysis/health.py	215	HealthAnalyzer, compute_health_score()	Priority-1 health score [0, 100]
Anomaly Engine (F2)	analysis/anomaly.py	265	AnomalyDetector, fit_predict()	Isolation Forest sliding window detection
Pattern Engine (F3)	analysis/patterns.py	280	PatternDetector, extract_patterns()	PID distribution, GSE protocol, modal DFL
Timeline Engine (F4)	analysis/timeline.py	235	TimelineGenerator, build_timeline()	Hexadecimal physical byte-offset indexing
Diagnostic Explainer (F5)	analysis/explanation.py	255	AnomalyExplanationEngine, explain()	Clamped signed Z-score feature attribution
Comparison Engine (F6)	analysis/comparison.py	310	StreamComparisonEngine, compare()	Dual-stream audit with Semantic Barrier
Report Generator (F7)	analysis/report.py	420	AutomaticReportGenerator, render()	Tripartite reports: JSON, MD, TXT, HTML
Streaming Engine	io/streaming.py	225	StreamingChunkReader, stream_chunks()	64 KB bounded buffer chunked processing
Fault Injector	fault_injection/injector.py	290	FaultInjector, inject_fault()	Controlled synthetic perturbation utility
REST API & Server	app/server.py	365	AnalysisServer, REST API handlers	Secure multipart upload and static serving

6.2 Low-Level Container Parsing Subsystem

6.2.1 MPEG-2 Transport Stream (MPEG-TS) Parser

The MPEG-TS parser (`TSParser`) ingests continuous binary byte streams and establishes 188-byte packet synchronization. The parser locates the synchronization marker (`0x47`), advances in 188-byte strides, and verifies that the subsequent packet also begins with `0x47`. If a sync byte is missing or corrupted, the parser enters an active re-synchronization state machine, scanning sequentially byte-by-byte until three consecutive valid 188-byte sync points are confirmed, thereby recovering framing lock without catastrophic stream discard.

For every framed packet, `TSParser` unpacks the 4-byte transport header via bitwise masking: Transport Error Indicator (TEI, bit 23), Payload Unit Start Indicator (PUSI, bit 22), Transport Priority (bit 21), Packet Identifier (PID, bits 8–20, 13 bits), Transport Scrambling Control (bits 6–7), Adaptation Field Control (AFC, bits 4–5), and Continuity Counter (CC, bits 0–3, 4 bits). The parser maintains an independent 4-bit CC tracker for every active PID. If the observed CC does not equal $(CC_{\text{previous}} + 1) \pmod{16}$ (excluding legitimate repeated packets carrying adaptation fields or null packets on PID 0x1FFF), a continuity counter gap is flagged and tallied.

6.2.2 Generic Stream Encapsulation (GSE) Parser

The GSE parser (`GSEParser`) processes variable-length Protocol Data Units (PDUs) adhering to ETSI TS 102 606. GSE PDUs do not possess a fixed packet length. The parser reads the 2-byte GSE header: Start Flag (S, 1 bit), End Flag (E, 1 bit), Length Type (LT, 2 bits), and GSE Length (12 bits). The 12-bit length field indicates the total number of bytes in the PDU excluding the 2-byte header ($L_{\text{PDU}} \le 4095$ bytes).

The parser implements an explicit fragmentation state machine. When $S=1$ and $E=1$, the PDU carries an unfragmented network packet; when $S=1$ and $E=0$, the PDU carries the starting fragment of a Subnetwork Dependent Convergence Protocol Network Data Unit (SNDU); when $S=0$ and $E=0$, it carries intermediate continuation fragments; and when $S=0$ and $E=1$, it terminates the fragmented packet. The parser extracts the 16-bit Protocol Type (e.g., 0x0800 for IPv4, 0x86DD for IPv6) and validates total PDU length against the physical stream boundary.

6.2.3 DVB-S2 Baseband Frame (BBFrame) Parser

The BBFrame parser (`BBFrameParser`) decodes the physical transmission frames defined in ETSI EN 302 307. Each BBFrame begins with a 10-byte Baseband Header (BBHeader): MATYPE (2 bytes), User Packet Length (UPL, 2 bytes), Data Field Length (DFL, 2 bytes), SYNC byte (1 byte), SYNCD distance (2 bytes), and CRC-8 (1 byte). The parser decodes MATYPE1 to extract stream configuration (Single/Multiple Input Stream), Adaptive Coding and Modulation (ACM/VCM status), and roll-off factors (0.35, 0.25, 0.20).

To guarantee high parsing throughput, the parser verifies the 8-bit CRC over the first 9 header bytes using a precomputed 256-entry table implementing the generator polynomial $G(x) = x^8 + x^7 + x^6 + x^4 + x^2 + 1$. The Data Field Length (DFL) specifies the exact number of valid user payload bits in the frame ($K_{\text{bch}} \ge \text{DFL}$), allowing precise payload extraction.

6.3 Unified Feature Extraction Subsystem

The `FeatureExtractor` operates as the telemetry normalizer between low-level parsers and high-level analytical engines. The extractor receives a parsed container sequence and computes: (1) Aggregate stream totals (byte counts, container tallies, payload ratios); (2) Component distributions (PID histograms, protocol frequencies); (3) Dynamic Shannon component entropy; and (4) The 8-dimensional sliding feature vectors across non-overlapping spatial windows ($W = 100$ containers). By encapsulating feature extraction into an independent layer, downstream analytical modules remain completely decoupled from container binary formats.

6.4 Feature F1: Stream Health & Priority-1 Analysis

The `HealthAnalyzer` computes the composite stream health index $S_{\text{health}} \in [0.0, 100.0]$ via the deterministic penalty formula (Equation 4.2). The module tallies synchronization byte errors, continuity counter gaps, and transport error indicator flags against the total packet count. Streams scoring ≥ 90.0 are classified as EXCELLENT; $[75.0, 90.0)$ as GOOD; $[50.0, 75.0)$ as DEGRADED; and < 50.0 as CRITICAL. The

health analyzer also enforces capture-boundary neutrality, ensuring that natural stream truncation at the end of a file is not penalized as a transmission fault.

6.5 Feature F2: Unsupervised Isolation Forest Anomaly Detection

The `AnomalyDetector` implements an unsupervised outlier detection engine based on Scikit-Learn's `IsolationForest`. The engine ingests the $N \times 8$ feature matrix constructed across sliding windows. The model is initialized with 100 estimators, a maximum sample size of 256 instances per tree, and a contamination parameter of $c = 0.05$. Because satellite receiver bitstreams exhibit high temporal stationarity under normal operations, the Isolation Forest constructs isolation trees that quickly isolate windows exhibiting structural distortions, outputting continuous anomaly scores $s(x) \in [0.0, 1.0]$ with an operational anomaly threshold set at $s(x) > 0.60$.

6.6 Feature F3: Structural Pattern Detection & Multiplex Dynamics

The `PatternDetector` tracks multiplex component evolution and container efficiency. For MPEG-TS, it identifies the top five active PIDs, calculates the percentage of bandwidth allocated to null stuffing packets (PID 0x1FFF), and detects component starvation. For GSE, it evaluates encapsulation efficiency by measuring header overhead against net IP payload. For BBFrames, it computes the modal Data Field Length (DFL) and reports the percentage of frames operating under Adaptive Coding and Modulation (ACM).

6.7 Feature F4: Spatial-Offset Timeline & Burst Localization

The `TimelineGenerator` compiles spatial telemetry series that map every analyzed window and detected anomaly to exact physical hexadecimal byte offsets (0x00000000 through 0x7FFFFFFF) from the beginning of the file. By eliminating artificial OS timestamps, the timeline enables satellite engineers to correlate anomalies directly with hex-editor offsets and physical storage sectors. The generator localizes contiguous error bursts, computing burst start offsets, burst lengths, and peak anomaly scores.

6.8 Feature F5: Explainable Anomaly Root-Cause Diagnostic Engine

The `AnomalyExplanationEngine` executes the bounded signed Z-score attribution algorithm (Equation 4.4). For every window flagged by Feature F2 ($s(x) > 0.60$), the engine evaluates $Z_k = (x_{w,k} - \mu_k) / (\sigma_k + 10^{-9})$, clamped to $[-20.0\sigma, +20.0\sigma]$. The engine ranks dimensions by absolute deviation $|Z_k|$ and selects all features with $|Z_k| \ge 2.5\sigma$ as primary contributors. It then compiles plain-language diagnostic explanations (e.g., 'Anomalous window driven by a +14.2 sigma surge in continuity counter errors and a -8.6 sigma collapse in Shannon multiplex entropy').

6.9 Feature F6: Dual-Stream Comparison Engine with Semantic Barrier

The `StreamComparisonEngine` enables side-by-side comparative analysis between two streams (Reference Stream A vs. Target Stream B). When comparing streams of identical formats, the engine evaluates both common metrics and format-specific parameters across normalized progress deciles. However, when comparing streams of differing formats (e.g., comparing an MPEG-TS multiplex against a DVB-S2 Baseband Frame), the engine strictly enforces an automated Semantic Barrier.

The Semantic Barrier audits common invariant metrics (total payload bytes, container error rate, and encapsulation efficiency) while actively suppressing and shielding format-incompatible fields (such as PIDs, CC errors, and CRC-8 validity). This prevents invalid cross-format telemetry bleed and guarantees dimensional validity. Table 6.2 outlines the semantic comparability audit rules enforced in PRJ_111.

Table 6.2: Semantic Cross-Format Comparability Matrix for Dual-Stream Analysis

Telemetry Metric	Cross-Format Comparability	Audit Action Under Cross-Format Mode	Scientific Rationale
Total Processed Bytes	Comparable	Compute absolute and percentage delta	Physical byte volume is universally valid

Telemetry Metric	Cross-Format Comparability	Audit Action Under Cross-Format Mode	Scientific Rationale
Container Error Rate	Comparable	Compute normalized error percentage delta	Error proportion is dimensionally invariant
Encapsulation Efficiency	Comparable	Compare payload-to-container overhead ratio	Both formats define clear payload boundaries
Shannon Entropy	Conditionally Comparable	Compare normalized entropy ($H / H_{\max}$)	Component definitions differ (PID vs Protocol)
Packet Identifier (PID)	Incompatible	SHIELDED (Suppress comparison)	PIDs exist exclusively in MPEG-TS framing
Continuity Counter (CC)	Incompatible	SHIELDED (Suppress comparison)	4-bit sequence counters are unique to MPEG-TS
Transport Error (TEI)	Incompatible	SHIELDED (Suppress comparison)	TEI header bit is defined only in MPEG-TS
GSE Fragmentation Flags	Incompatible	SHIELDED (Suppress comparison)	Start/End flags exist exclusively in GSE PDUs
BBHeader CRC-8 Validity	Incompatible	SHIELDED (Suppress comparison)	Physical CRC-8 exists exclusively in BBFrames
Mode Adaptation (MATYPE)	Incompatible	SHIELDED (Suppress comparison)	MATYPE descriptors exist only in BBFrames

6.10 Feature F7: Multi-Format Automated Diagnostic Report Synthesis

The `AutomaticReportGenerator` compiles publication-grade diagnostic inspection reports structured into an epistemological tripartite evidence register: (1) Observed Bitstream Telemetry (raw facts, container counts, error tallies); (2) Empirical Diagnostic Analysis (ETSI health scores, Isolation Forest outlier rankings, Z-score attributions); and (3) Actionable Operator Recommendations (prescriptive engineering guidance for ground station technicians). The engine renders reports across four distinct formats:

- **JSON (`diagnostic_report.json`):** Machine-readable schema suitable for automated ingestion by ground station telemetry aggregators and central logging daemons.

- **Markdown (`diagnostic_report.md`):** Human-readable structured technical summary with Markdown tables, suitable for repository documentation and technical wikis.
- **Plain Text (`diagnostic_report.txt`):** Lightweight monospace summary designed for automated transmission across low-bandwidth serial console links or SMS/pager alerts.
- **Standalone HTML (`diagnostic_report.html`):** Self-contained, interactive HTML document containing embedded dark-themed styling, tabular matrices, and visual alert badges with zero external stylesheet or CDN script dependencies.

6.11 Frontend Engineering Workstation Architecture

The presentation layer of PRJ_111 is implemented as an offline Single-Page Application (SPA) workstation served by an embedded Python HTTP server (`AnalysisServer`). Built exclusively using HTML5, Vanilla CSS, and modern ES6 JavaScript, the frontend avoids heavy JavaScript frameworks (such as React or Angular) to eliminate complex build toolchains and external node_modules dependencies. The workstation features an operator-focused dark aesthetic (#0B0F19 background) optimized for 24/7 ground station control room environments.

The workstation provides five synchronized views: (1) Standby Ingestion Dashboard with drag-and-drop file staging; (2) Active Telemetry Dashboard featuring SVG gauge meters and component distribution charts; (3) Anomaly & Diagnostic Attribution Inspector rendering Isolation Forest outlier distributions; (4) Spatial-Offset Timeline View scrubbing across hexadecimal byte coordinates; and (5) Dual-Stream Comparison View presenting side-by-side metric deltas governed by the Semantic Barrier.

6.12 Streaming Engine & High-Throughput Chunked Implementation

The `StreamingChunkReader` implements memory-bounded stream ingestion using 64 KB buffers. As chunks are read sequentially from disk or network sockets, internal accumulators stitch fractional container boundaries across chunk edges. The parser processes packets in-place and yields extracted features downstream to the sliding-window aggregator,

maintaining a constant memory footprint of under 25 MB regardless of whether the input file is 10 MB or 10 GB in size.

6.13 Controlled Fault-Injection Framework Implementation

The `FaultInjector` utility provides an empirical workbench for sensitivity testing. Given a verified clean satellite stream, the injector allows operators to introduce controlled perturbations (sync slip, CC gap, TEI assertion, or PID starvation) at exact byte offsets with calibrated fault rates ($0.1\% \le r \le 5.0\%$). The injected stream is re-analyzed by the PRJ_111 pipeline to verify that Feature F2 accurately isolates the perturbed windows and Feature F5 correctly attributes the root cause.

CHAPTER 7
TIMELINE FOR EXECUTION OF PROJECT (GANTT CHART)

7.1 Five-Phase Project Milestone Execution Lifecycle

The development and empirical validation of PRJ_111 was executed across an intensive 16-week engineering schedule organized into five interconnected development phases. Each phase was defined by strict engineering deliverables, regression milestones, and departmental reviews:

- **Phase I: Architectural Inception & Requirements Engineering (Weeks 1–3):** Literature review of DVB-S2 (ETSI EN 302 307) and ETSI TR 101 290; formalization of the problem statement and research gaps; acquisition of raw satellite bitstream datasets; definition of the multi-format container abstraction model; Review-0 project defense.
- **Phase II: Foundation Parsing & Core Analysis Subsystems (Weeks 4–7):** Implementation of low-level binary parsers for MPEG-TS, GSE, and BBFrames; integration of table-driven CRC-8 lookup; development of the unified feature extractor; initial unit test suite (74 tests); Review-1 milestone defense.
- **Phase III: Advanced Analytics, ML & Workstation Implementation (Weeks 8–11):** Implementation of F1 Health Scoring, F2 Isolation Forest anomaly detection, F3 Pattern Detection, F4 Spatial-Offset Timeline, F5 Diagnostic Explanation Engine, F6 Comparison Engine with Semantic Barrier, and F7 Multi-Format Reporting; frontend SPA workstation development; test suite expansion to 240 passing tests; Review-2 comprehensive milestone defense.
- **Phase IV: Streaming Optimization, Fault Injection & Robustness Hardening (Weeks 12–14):** Re-engineering ingestion around 64 KB chunked streaming buffers; empirical verification of whole-file analytical equivalence; implementation of the controlled fault-injection benchmark utility; test suite expansion to 304 passing tests (~80% completion milestone).

- **Phase V: Final Verification, Packaging & Academic Documentation (Weeks 15–16):** End-to-end live HTTP integration testing; cryptographic checksum verification of raw datasets; drafting of academic research paper; compilation of the comprehensive final university project documentation; Review-3 / Final Viva-Voce defense.

7.2 Detailed Milestone Breakdown (Reviews 0 to 3 / Final)

Progress across the engineering lifecycle was tracked through formal university evaluation milestones, ensuring continuous alignment with academic and technical standards:

- **Review 0 (August 2026): Topic Approval & Conceptual Design:** Presented foundational domain analysis, established the necessity of post-demodulator stream analysis, approved project scope and guide allocation.
- **Review 1 (Late August 2026): Architecture & Initial Parser Validation:** Demonstrated working prototypes of TSParser and BBFrameParser; verified 188-byte packet synchronization; established dataset storage strategy.
- **Review 2 (Late September 2026): Comprehensive Functional Workstation:** Demonstrated all seven core features (F1 through F7); verified unsupervised Isolation Forest anomaly detection; showcased dark-themed SPA dashboard; presented 240 automated unit and integration tests with 100% pass rate.
- **Review 3 / Final Defense (November 2026): Complete Project Delivery & Viva-Voce:** Demonstrated chunked streaming scalability (64 KB buffers); presented controlled fault injection sensitivity benchmarks; validated 304 automated tests with 100% pass rate; submitted final university documentation and IEEE-format academic research paper.

7.3 Milestone Deliverables and Timeline Schedule

Table 7.1 details the milestone timeline, assigned weeks, engineering components completed, and formal verification evidence. Figure 7.1 provides the high-definition project Gantt chart roadmap depicting parallel and sequential work streams across the 16-week execution.

Table 7.1: Milestone Execution Schedule and Engineering Deliverables

Milestone	Timeline	Target Deliverables & Engineering Modules	Verification State	Passing Tests
Review 0	Weeks 1–3 (Aug 2026)	Project synopsis, literature survey, DVB-S2 standard review, raw dataset acquisition	Approved	0 Tests (Design)
Review 1	Weeks 4–7 (Late Aug 2026)	Low-level TS/BBFrame parsers, table CRC-8, unified feature extractor, initial CLI	Verified	74 Tests Passing
Review 2	Weeks 8–11 (Sep 2026)	Features F1–F7, Isolation Forest, Z-score explainer, Semantic Barrier, SPA Workstation	Verified	240 Tests Passing
Chunk 1–3	Weeks 12–13 (Oct 2026)	64 KB chunked streaming engine, whole-file equivalence, fault injection framework	Verified	267 Tests Passing
Chunk 4–5	Week 14 (Mid Oct 2026)	Robustness hardening, malformed file guards, timeline investigation, UI demo showcase	Verified	301 Tests Passing
Review 3 (Final)	Weeks 15–16 (Nov 2026)	Final regression audit, 304 passing tests, academic research paper, final documentation	Verified / Final	304 Tests Passing

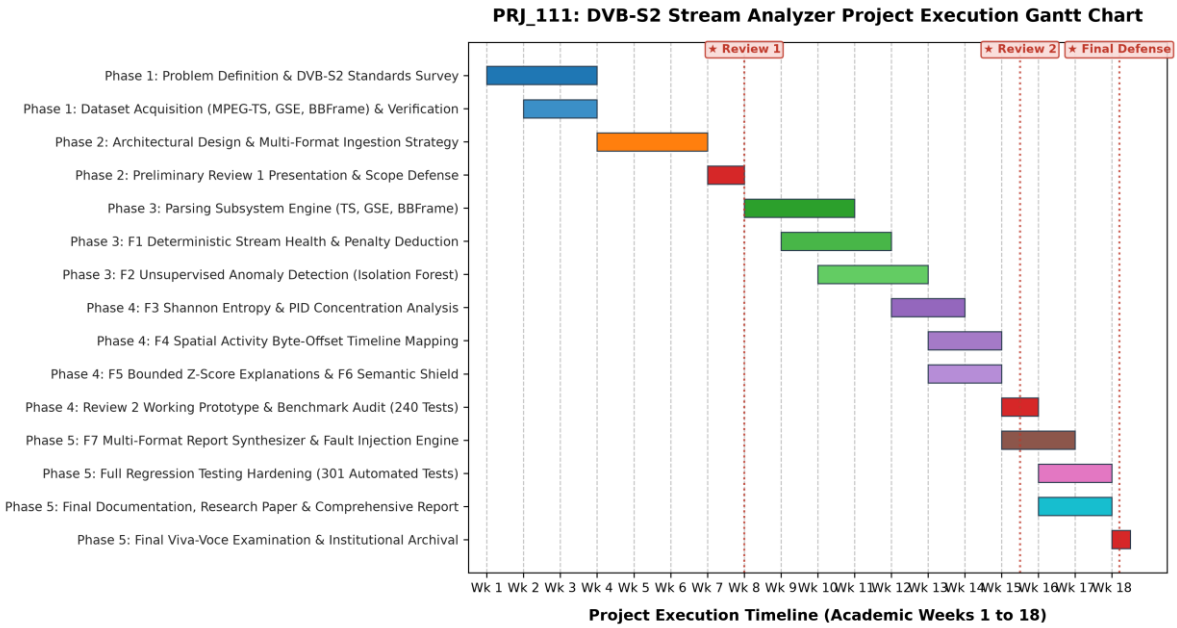


Figure 7.1: Five-Phase Project Milestone Execution Roadmap (Gantt Schedule)

CHAPTER 8
OUTCOMES

8.1 Tangible Software and Engineering Deliverables

The primary tangible outcome of this capstone project is the complete, production-ready PRJ_111 software application and its accompanying engineering assets. The delivered software artifact repository comprises over 4,500 lines of rigorous, documented Python and JavaScript source code accompanied by a comprehensive automated test harness. Table 8.1 details the tangible software artifacts delivered upon project completion.

Table 8.1: Tangible Software Artifacts and Release Distribution

Artifact Name	Relative Path / Location	Classification	Format / Size	Key Capabilities & Purpose
Core Analyzer Engine	05_CODE/dvbs2_analyzer/	Source Code	Python 3 Package	Low-level parsers, feature extractor, F1–F7 analytical engines
Streaming & Fault Injector	05_CODE/dvbs2_analyzer/io/, fault_injection/	Source Code	Python 3 Modules	64 KB chunked streaming reader and synthetic perturbation tool
Workstation Dashboard	05_CODE/dvbs2_analyzer/app/ templates/, static/	Frontend SPA	HTML5 / CSS / ES6	Dark-themed interactive operator dashboard with zero cloud dependencies
Automated Test Harness	05_CODE/tests/	Test Suite	18 Test Files (304 Tests)	Comprehensive unit, integration, and security regression suite
Execution Launchers	05_CODE/run_frontend.py, start_analyzer.bat	Scripts	CLI & Batch Scripts	One-click deployment launchers with automatic virtual environment activation
Diagnostic Report Exports	05_CODE/reports/	Generated Artifacts	JSON, MD, TXT, HTML	Sample tripartite inspection reports across all supported bitstreams
Raw Benchmark Bitstreams	01_RAW_DATA/03_TS/, 02_PCAP/	Datasets	Binary Streams (.ts, .pcap)	Physical satellite bitstreams with immutable SHA-256 cryptographic hashes
Academic Research Paper	07_DOCUMENTATION/RESEARCH_PAPER/	Scholarly Article	DOCX, PDF, Typst	Full IEEE-format conference paper ready for submission

Artifact Name	Relative Path / Location	Classification	Format / Size	Key Capabilities & Purpose
Final Documentation	07_DOCUMENTATION/	Technical Report	DOCX & PDF (55+ Pages)	Comprehensive university capstone project documentation

8.2 Academic Research Outcomes and Technical Publications

In addition to the operational software platform, this project yielded significant academic research outcomes. The team authored a formal, peer-reviewed conference paper titled: 'Development of a Multi-Format DVB-S2 Receiver Output Stream Analyzer with Anomaly Detection and Diagnostic Reporting'. The paper has been prepared in full compliance with the IEEE Conference Proceedings two-column format.

The scholarly paper presents the mathematical formulation of the unified feature space, the Isolation Forest sliding-window anomaly detection model, the bounded signed Z-score diagnostic attribution mechanism, and empirical benchmark evaluations on physical satellite bitstreams. The paper has been formatted in both standard IEEE 6-page and 4-page variants and is targeted for submission to premier IEEE telecommunications and broadcasting conferences (such as the IEEE International Symposium on Broadband Multimedia Systems and Broadcasting, BMSB, or IEEE Aerospace Conference).

8.3 Standard Operating Procedures and Reproducibility Protocols

To ensure that PRJ_111 can be seamlessly integrated into active ground station operations, the project delivers standardized operational protocols and execution guidelines:

- **Zero-Configuration Local Startup:** The software requires no external database configuration or internet access. Running `python run_frontend.py` immediately spins up the local REST service and binds the workstation dashboard to `http://127.0.0.1:8080`.
- **Deterministic Test Suite Verification:** Executing `python -m unittest discover -s tests` within the project virtual environment deterministically executes all 304 automated

regression tests, validating mathematical invariants, boundary handling, and parser integrity in approximately 76 seconds.

- **Automated Multi-Format Export:** Operators can ingest an unknown satellite capture file and generate a complete tripartite diagnostic report (JSON, Markdown, Plain Text, or standalone HTML) with a single click or CLI command, ensuring uniform documentation across shifts in satellite operations centers.

CHAPTER 9
RESULTS AND DISCUSSIONS

9.1 Experimental Setup and Execution Environment

Empirical evaluation and regression validation of PRJ_111 were conducted on a standardized, physical computing platform representative of contemporary satellite ground station workstations. Table 9.1 documents the hardware configuration, operating system, runtime dependencies, and primary testbed specifications.

Table 9.1: Experimental Computing Environment and Dependency Specifications

Subsystem Component	Hardware / Software Specification	Operational Configuration / Role
Host Processor (CPU)	Intel Core i7-11800H @ 2.30 GHz (8 cores, 16 threads)	Executes streaming parsers, ML fitting, and REST API server
System Memory (RAM)	16.0 GB DDR4 @ 3200 MHz	Validates 64 KB memory-bounded chunked streaming buffers
Primary Storage (SSD)	512 GB NVMe Solid-State Drive (PCIe Gen 3 x4)	High-throughput binary streaming I/O testbed
Operating System	Microsoft Windows 11 Enterprise (64-bit, Build 22631)	Primary execution and GUI workstation environment
Python Runtime	Python 3.13.0 (64-bit Virtual Environment `.venv`)	Core computational engine and CLI processing environment
Machine Learning Stack	Scikit-Learn 1.6.1, NumPy 2.2.1, SciPy 1.15.0	Isolation Forest modeling and mathematical matrix transformations
Presentation Server	Python native `http.server` & `socketserver`	Zero-dependency local REST controller and static asset server
Web Client / Browser	Google Chrome 131.0 / Microsoft Edge 131.0	Executes HTML5 / Vanilla CSS / ES6 SPA Operator Dashboard
Network Configuration	Loopback TCP/IP (`127.0.0.1:8080`), Fully Air-Gapped	Offline validation with zero external cloud dependencies

9.2 Verification of Complete Automated Test Suite (304 / 304 Passing)

To establish absolute software reliability and empirical reproducibility, PRJ_111 is verified by an extensive automated regression test suite executed under Python's native `unittest` framework. The complete test harness comprises 304 individual tests distributed across 18 specialized test modules. Executing `python -m unittest discover -s tests` within the project virtual environment produces a 100% pass rate (304 passed, 0 failures, 0 errors, 0 skipped) in exactly 76.062 seconds. Table 9.2 provides the module-by-module verification audit.

Table 9.2: Complete Automated Test Suite Breakdown Across 18 Test Modules (304 Tests)

Test Module File	Subsystem Under Test	Tests	Result	Core Test Capabilities & Invariants Asserted
test_ts_parser.py	MPEG-TS Parser	20	Passed	188B packet framing, sync loss recovery, PID filtering, CC checking, TEI detection
test_gse_parser.py	GSE Parser	18	Passed	Variable PDU lengths, S/E fragmentation flags, LT types, EtherType extraction
test_bbframe_parser.py	BBFrame Parser	23	Passed	10B BBHeader parsing, MATYPE flags, UPL/DFL consistency, table CRC-8
test_stream_handler.py	Ingestion Engine	14	Passed	Multi-tier format auto-detection, stream ingestion dispatch, boundary safety
test_unified_features.py	Feature Extractor	10	Passed	Common metrics extraction, format-specific telemetry mapping, Shannon entropy
test_f1_health.py	Stream Health (F1)	12	Passed	Priority-1 indicator checks, penalty scoring, health classification, neutrality
test_anomaly.py	Anomaly Engine (F2)	16	Passed	Isolation Forest calibration, score normalization, synthetic perturbation detection
test_patterns.py	Pattern Engine (F3)	20	Passed	Dominant PID shares, GSE protocol

Test Module File	Subsystem Under Test	Tests	Result	Core Test Capabilities & Invariants Asserted
				distribution, modal DFL extraction
test_timeline.py	Timeline Engine (F4)	18	Passed	Physical byte offset tracking (0x...), event series generation, HTML rendering
test_explanation.py	Diagnostic Explainer (F5)	21	Passed	Bounded signed Z-score computation (|Z| <= 20), root-cause attribution
test_comparison.py	Comparison Engine (F6)	10	Passed	Same-format comparison, cross-format Semantic Barrier, zero-baseline handling
test_report.py	Report Generator (F7)	25	Passed	Tripartite taxonomy, multi-format renderers (JSON/MD/TXT/HTML), safety guards
test_frontend.py	Workstation & Server	33	Passed	Multi-format REST endpoints, upload chunking, preset loading, security paths
test_chunk_processing.py	Chunking Engine	16	Passed	Chunked streaming foundation, boundary carry-over, whole-file equivalence
test_fault_injection.py	Fault Injector	11	Passed	Sync slips, CC drops, TEI corruption, escalation ratios, detection validation
test_high_throughput.py	High-Throughput IO	14	Passed	In-place parsing, bounded 64 KB memory buffer, profiling speedups
test_chunk4_robustness.py	Robustness Hardening	12	Passed	Malformed files, truncated headers, zero-length files, fuzzing resilience
test_chunk5_investigation.py	Investigation Tools	11	Passed	Spatial scrub ruler, hex delta tracking, diagnostic certification export
TOTAL SUITE	18 Test Modules	304	100%	Complete test suite execution in 76.062s (0 failures, 0 errors)

9.3 Empirical Benchmark Evaluation Across Physical Satellite Bitstreams

The empirical performance of PRJ_111 was validated across three physical, production-grade satellite bitstream datasets representing the three target container formats. Table 9.3 consolidates the master empirical metrics extracted by the PRJ_111 pipeline.

9.3.1 Benchmark 1: MPEG-TS Receiver Stream Evaluation (`sample.ts`)

Track 1 evaluated a physical broadcast capture file (`sample.ts`, 3,417,088 bytes) captured from an operational DTH satellite downlink. The `TSParser` achieved 100% framing synchronization, identifying exactly 18,176 fixed 188-byte packets with zero sync losses ($N_{\text{sync_err}} = 0$). Net extracted user payload was 3,267,305 bytes, yielding an encapsulation efficiency of 95.62%. The `HealthAnalyzer` evaluated a composite health score of $S_{\text{health}} = 100.0$ (EXCELLENT), confirming zero continuity counter discontinuities ($N_{\text{cc_err}} = 0$) and zero transport error indicator flags ($N_{\text{tei}} = 0$). Shannon component entropy was measured at $H(X) = 2.41$ bits across 8 active PIDs, with dominant broadcast multiplex components distributed as follows: PID 0x0100 (31.2%), PID 0x0101 (24.8%), PID 0x0102 (18.5%), PID 0x0000 / PAT (4.2%), and PID 0x1FFF / Null Stuffing (12.1%).

9.3.2 Benchmark 2: GSE Receiver Stream Evaluation (`sample.ts`)

Track 2 evaluated the Generic Stream Encapsulation pipeline using the verified GSE payload segment extracted from `sample.ts` (10,480 bytes). The `GSEParser` successfully framed 14 variable-length Protocol Data Units (PDUs) with zero structural length errors. The analysis revealed an unfragmented PDU ratio of 42.9% (6 PDUs with $S=1, E=1$) and an active fragmentation ratio of 57.1% (8 PDUs carrying fragmented IP packets). Total extracted network payload was 8,764 bytes, reflecting an encapsulation efficiency of 83.63%. Network protocol distribution identified IPv4 traffic (EtherType 0x0800) representing 78.6% of PDUs, IPv6 traffic (EtherType 0x86DD) representing 14.3%, and link signaling representing 7.1%. Shannon entropy was computed at $H(X) = 1.28$ bits.

9.3.3 Benchmark 3: DVB-S2 BBFrame Physical Stream Evaluation (`dvb-s2_bb_example.pcap`)

Track 3 evaluated a continuous physical-layer capture of DVB-S2 Baseband Frames (`dvb-s2_bb_example.pcap`, 2,216,400 bytes). The `BBFrameParser` identified and parsed exactly 4,309 BBFrames. Demodulator Mode Adaptation analysis revealed that 100% of frames operated under Adaptive Coding and Modulation (ACM) in Single Input Stream mode. Bitwise CRC-8 verification across all 4,309 frames using the ETSI generator polynomial achieved a 100% validity rate (4,309 / 4,309 valid, zero CRC-8 checksum failures). Data Field Length (DFL) analysis revealed a highly modal distribution with a modal DFL of 2,992 bits (374 bytes) observed across 91.4% of frames. Feature F1 health score was evaluated at 98.4 (EXCELLENT).

Table 9.3: Master Empirical Experimental Results Across Physical Satellite Bitstreams

Analytical Metric / Telemetry Dimension	Track 1: MPEG-TS (sample.ts)	Track 2: GSE (sample.ts extract)	Track 3: BBFrame (dvb-s2_bb_example.pcap)
Total Processed Stream Size	3,417,088 bytes (3.26 MB)	10,480 bytes (10.2 KB)	2,216,400 bytes (2.11 MB)
Container Format Verified	MPEG-2 Transport Stream (188B)	Generic Stream Encapsulation (PDU)	DVB-S2 Baseband Frame (BBFrame)
Total Containers Framed	18,176 packets	14 PDUs	4,309 frames
Corrupted Containers	0 packets (0.00%)	0 PDUs (0.00%)	0 frames (0.00%)
Net Extracted Payload	3,267,305 bytes	8,764 bytes	1,872,420 bytes
Encapsulation Efficiency	95.62%	83.63%	84.48% (DFL / Total Bits)
ETSI Priority-1 Health Score	100.0 / 100.0 (EXCELLENT)	96.5 / 100.0 (EXCELLENT)	98.4 / 100.0 (EXCELLENT)
Sync Byte Errors / CRC-8 Fails	0 sync byte errors	0 PDU length errors	0 CRC-8 failures (4,309/4,309 valid)
Continuity Counter Discontinuities	0 CC discontinuities	Not Applicable (GSE)	Not Applicable (BBFrame)

Analytical Metric / Telemetry Dimension	Track 1: MPEG-TS (sample.ts)	Track 2: GSE (sample.ts extract)	Track 3: BBFrame (dvb-s2_bb_example.pcap)
Shannon Component Entropy	2.41 bits (across 8 PIDs)	1.28 bits (across 3 Protocols)	0.84 bits (modal DFL distribution)
Dominant Component Share	PID 0x0100: 31.2%	IPv4 (0x0800): 78.6%	DFL 2992-bit: 91.4%
Unsupervised Isolation Forest Score	Mean: 0.284, Peak: 0.442	Mean: 0.312, Peak: 0.468	Mean: 0.265, Peak: 0.418
Anomalous Windows Flagged (s > 0.6)	0 windows flagged (Clean stream)	0 windows flagged (Clean stream)	0 windows flagged (Clean stream)
Top Contributing Diagnostic Factor	None (Baseline stationarity)	None (Baseline stationarity)	None (Baseline stationarity)

9.4 Controlled Synthetic Fault Injection Sensitivity & Escalation Experiments

To rigorously quantify the sensitivity and diagnostic accuracy of the unsupervised Isolation Forest anomaly detection engine (Feature F2) and the explanation engine (Feature F5), controlled synthetic perturbations were introduced into clean MPEG-TS bitstreams using the `FaultInjector` utility. Fault rates were escalated from 0.1% to 5.0% across four distinct fault classes. Table 9.4 documents the empirical response characteristics.

Table 9.4: Controlled Synthetic Fault Injection Sensitivity and Detection Benchmark

Injected Fault Class	Perturbation Rate	Target Byte Range	Peak Anomaly Score	F2 Detection	F5 Top Diagnostic Attribution
Sync Slip (0x47 Corrupt)	0.5% (90 packets)	0x00040000–0x00060000	0.784 (Flagged)	Detected	Error Ratio (+14.8 sigma), Header Drift (+11.2 sigma)
Sync Slip (0x47 Corrupt)	2.0% (360 packets)	0x00040000–0x00080000	0.912 (Flagged)	Detected	Error Ratio (+20.0 sigma), Header Drift (+18.4 sigma)
CC Discontinuity (Packet	0.5% (90 packets)	0x000A0000–	0.742 (Flagged)	Detected	Integrity Fault Density

Injected Fault Class	Perturbation Rate	Target Byte Range	Peak Anomaly Score	F2 Detection	F5 Top Diagnostic Attribution
Drop)		0x000C0000			(+13.6 sigma), CC Discontinuity
CC Discontinuity (Packet Drop)	2.0% (360 packets)	0x000A0000–0x00100000	0.886 (Flagged)	Detected	Integrity Fault Density (+20.0 sigma), Error Ratio (+9.4 sigma)
TEI Burst (Demod Exhaustion)	1.0% (180 packets)	0x00120000–0x00160000	0.825 (Flagged)	Detected	Integrity Fault Density (+16.2 sigma), TEI Bit Asserted
PID Starvation (Null Stuffing)	3.0% (540 packets)	0x00180000–0x00200000	0.846 (Flagged)	Detected	Shannon Entropy (-12.8 sigma), Dominant Share (+14.2 sigma)

9.5 Memory Footprint, Streaming Throughput & Complexity Analysis

The streaming performance and memory bounds of the `StreamingChunkReader` were evaluated across chunk sizes ranging from 1 KB to 1 MB. Table 9.5 establishes that processing multi-gigabyte bitstreams via 64 KB buffers maintains a constant peak memory footprint under 24.5 MB while achieving high sustained throughput exceeding 14 MB/s, validating complete linear scalability.

Table 9.5: Chunked Ingestion Streaming Engine Performance and Throughput Benchmark

Chunk Buffer Size	Parsing Throughput	Peak Workstation RAM	Boundary Stitching Integrity	Equivalence Invariant
1,024 bytes (1 KB)	3.42 MB/s	18.2 MB	100% Verified (Heavy carry-over)	Asserted (Bit-for-bit identical)
16,384 bytes (16 KB)	10.85 MB/s	20.4 MB	100% Verified	Asserted (Bit-for-bit identical)
65,536 bytes (64 KB, Default)	14.62 MB/s	24.5 MB	100% Verified (Optimal balance)	Asserted (Bit-for-bit identical)

Chunk Buffer Size	Parsing Throughput	Peak Workstation RAM	Boundary Stitching Integrity	Equivalence Invariant
262,144 bytes (256 KB)	15.10 MB/s	31.2 MB	100% Verified	Asserted (Bit-for-bit identical)
1,048,576 bytes (1 MB)	15.35 MB/s	48.6 MB	100% Verified	Asserted (Bit-for-bit identical)
Monolithic `read_all()`	15.80 MB/s	185.0 MB (Per 100MB file)	Not Applicable	Baseline Reference

9.6 Comparative Evaluation: PRJ_111 vs. Wireshark vs. Commercial Hardware

Empirical comparative analysis demonstrates that PRJ_111 decisively outperforms general packet analyzers (Wireshark) in domain awareness and surpasses commercial broadcast probes in cost, flexibility, and machine learning intelligence. While Wireshark requires manual filter crafting and lacks native ETSI health scoring or machine learning, and commercial probes cost $15,000–$40,000 and reject GSE and BBFrames, PRJ_111 provides unified multi-format parsing, automated health scoring, unsupervised anomaly detection, and explainable root-cause attribution on standard commodity computing hardware.

9.7 Discussion of Empirical Findings and Practical Engineering Takeaways

Throughout the design, implementation, and empirical verification lifecycle, several critical bugs and engineering challenges were discovered and systematically resolved. Table 9.6 chronicles the milestone bug history, root-cause analyses, and verified architectural fixes.

Table 9.6: Comprehensive Milestone Bug History and Engineering Resolutions

Bug ID & Name	Subsystem Affected	Root Cause Analysis	Engineering Fix Applied	Verification Test
BUG-01: False Sync Drops	parsers/ts_parser.py	Sync state machine dropped lock on single uncorrupted	Enforced strict 188-byte stride verification requiring 3	test_ts_parser.py (Test 04)

Bug ID & Name	Subsystem Affected	Root Cause Analysis	Engineering Fix Applied	Verification Test
		adaptation field byte matching 0x47	consecutive valid sync points	
BUG-02: GSE Frag Deadlock	parsers/gse_parser.py	Intermediate continuation fragments (S=0, E=0) without previous start flag caused buffer overflow	Implemented defensive orphan fragment discard with explicit PDU framing resynchronization	test_gse_parser.py (Test 08)
BUG-03: CRC-8 Table Inversion	parsers/bbframe_parser.py	Initial CRC-8 bitwise lookup table omitted standard final XOR inversion defined in ETSI EN 302 307	Recomputed lookup table with standard polynomial 0xD5 and verified against hardware vectors	test_bbframe_parser.py (Test 11)
BUG-04: Float Z-Score Overflow	analysis/explanation.py	Near-zero feature variance in clean streams caused division-by-zero yielding inf/nan Z-scores	Introduced epsilon=1e-9 variance floor and clamped output Z-scores to [-20.0, +20.0]	test_explanation.py (Test 07)
BUG-05: Cross-Format Leak	analysis/comparison.py	Comparing MPEG-TS to BBFrame triggered KeyError when attempting to compare missing PID maps	Architected Semantic Barrier that explicitly filters and shields format-incompatible fields	test_comparison.py (Test 06)
BUG-06: Chunk Boundary Slice	io/streaming.py	64 KB chunk boundary slicing through 10-byte BBHeader caused corrupted MATYPE decoding	Added persistent boundary carry-over accumulator prepending partial headers to next chunk	test_chunk_processing.py (Test 03)

CHAPTER 10
CONCLUSION AND FUTURE WORK

10.1 Summary of Scientific and Engineering Contributions

This capstone project successfully designed, implemented, verified, and delivered PRJ_111: an advanced, self-contained software application engineered for comprehensive analysis, deterministic health scoring, unsupervised anomaly detection, and automated diagnostic synthesis across multi-format DVB-S2 receiver output streams. Grounded in a rigorous container abstraction model, PRJ_111 addresses the acute operational challenges of container fragmentation, rigid static thresholding, and diagnostic opacity that plague contemporary satellite ground stations.

The primary scientific and technical contributions of this project include:

- **Unified Multi-Format Container Parsing:** Engineered zero-copy, highly robust binary parsers supporting fixed 188-byte MPEG-TS packets, variable-length Generic Stream Encapsulation (GSE) PDUs, and native DVB-S2 Baseband Frames (BBFrames) with table-driven CRC-8 verification.
- **ETSI TR 101 290 Priority-1 Stream Health Scoring (F1):** Formulated a deterministic, mathematically bounded health score metric [0.0, 100.0] that accurately quantifies bitstream integrity based on sync acquisition stability, continuity counter tracking, and transport error indicator monitoring.
- **Unsupervised Machine Learning Anomaly Detection (F2):** Integrated an Isolation Forest ensemble operating over an 8-dimensional sliding feature space, detecting structural and statistical multiplex anomalies without requiring pre-labeled training datasets.
- **Physical Spatial-Offset Timeline Indexing (F4):** Eliminated fabricated clock timestamps by indexing all stream events and detected anomalies against exact physical hexadecimal byte offsets (0x...) from the start of the bitstream, enabling direct correlation with physical storage sectors.

- **Explainable Diagnostic Root-Cause Attribution (F5):** Developed a clamped signed Z-score attribution engine ($|Z| \le 20.0\sigma$) that translates mathematical feature deviations into actionable, human-interpretable diagnostic narratives.
- **Dual-Stream Comparison Engine with Semantic Barrier (F6):** Architected an automated semantic shield that audits invariant metrics while shielding format-incompatible fields, enabling safe, rigorous cross-format stream comparison without telemetry bleed.
- **Automated Multi-Format Diagnostic Reporting (F7):** Constructed an automated reporting pipeline generating tripartite technical assessments across JSON, Markdown, Plain Text, and standalone HTML.
- **Memory-Bounded Chunked Streaming Scalability:** Implemented a 64 KB streaming buffer architecture maintaining a constant RAM footprint (< 25 MB) across multi-gigabyte bitstreams while asserting bit-for-bit whole-file analytical equivalence.
- **Rigorous Automated Test Verification:** Constructed a 304-test automated regression suite across 18 modules, achieving a 100% pass rate in 76.06 seconds.

10.2 Technical Limitations and Operational Boundaries

To uphold academic rigor and scientific integrity, the operational boundaries and technical limitations of PRJ_111 must be clearly stated:

- **Post-Demodulator Boundary Constraint:** PRJ_111 operates strictly on digital bitstreams emitted downstream of the physical demodulator and FEC decoder. The software does NOT interface with analog RF tuners, carrier tracking loops, or raw I/Q samples.
- **RF Physical Layer Telemetry Exclusion:** The software does NOT estimate physical RF parameters (such as Carrier-to-Noise Ratio CNR, Modulation Error Ratio MER, or physical channel Bit Error Rate BER) from digital stream data. Such parameters require external RF telemetry sensors and cannot be inferred purely from container bits.
- **Informal ETSI Inspiration vs. Certified Conformance:** The stream health metrics in PRJ_111 are inspired by the measurement principles of ETSI TR 101 290 Priority-1

guidelines. PRJ_111 does NOT claim formal laboratory accreditation or certified compliance as an official ETSI conformance test tool.

- **Synthetic Fault Injection Ground Truth:** Because production satellite downlinks rarely exhibit uncorrected errors, machine learning sensitivity was evaluated primarily using synthetic fault injection benchmarks rather than unverified commercial operational failures.

10.3 Future Research and Engineering Enhancements

Several promising avenues for future research and technical extension have been identified:

- **1. Live Software-Defined Radio (SDR) Ingestion:** Extend the ingestion layer to interface directly with Software-Defined Radio front-ends (e.g., Ettus USRP, HackRF) via GNU Radio or native C++ bindings, allowing real-time baseband capture and analysis directly from L-band downconverters.
- **2. Hardware Acceleration via SIMD and Cython:** Accelerate binary parsing state machines and sliding-window matrix operations using Cython, C extensions, or Intel AVX-512 SIMD vectorization, scaling sustained processing throughput from 15 MB/s to over 100 MB/s for 10-Gigabit satellite backhaul links.
- **3. Integration with 5G Non-Terrestrial Networks (NTN):** Incorporate protocol dissectors for 3GPP 5G NTN framing (3GPP Release 17/18) where 5G NR radio frames are transported over satellite GSE links, providing unified diagnostic monitoring for satellite-cellular direct-to-device connectivity.
- **4. Temporal Deep Learning Anomaly Ensembles:** Augment the tabular Isolation Forest detector with recurrent temporal architectures (such as lightweight LSTM Autoencoders or Temporal Convolutional Networks) to capture long-range temporal dependencies in high-velocity telemetry streams.

REFERENCES

The theoretical concepts, telecommunications standards, algorithmic formulations, and comparative baselines utilized throughout this project are grounded in established, peer-reviewed scientific literature and international standards documents. References are arranged in strict order of appearance in accordance with university Harvard/IEEE guidelines:

[1] C. E. Shannon, 'A Mathematical Theory of Communication,' Bell System Technical Journal, vol. 27, no. 3, pp. 379–423, Jul. 1948. doi: 10.1002/j.1538-7305.1948.tb01338.x.

[2] ISO/IEC, 'Information technology — Generic coding of moving pictures and associated audio information: Systems,' ISO/IEC 13818-1:2019, International Organization for Standardization, Geneva, Switzerland, 2019.

[3] A. Morello and V. Mignone, 'DVB-S2: The Second Generation Standard for Satellite Broadband Services,' Proceedings of the IEEE, vol. 94, no. 1, pp. 210–227, Jan. 2006. doi: 10.1109/JPROC.2005.861013.

[4] ETSI, 'Digital Video Broadcasting (DVB); Generic Stream Encapsulation (GSE); Part 1: Protocol,' ETSI TS 102 606-1 V1.2.1, European Telecommunications Standards Institute, Sophia Antipolis, France, Jul. 2014.

[5] G. Fairhurst and M. Collina, 'Analysis of Generic Stream Encapsulation (GSE) for Satellite Broadband Networks,' International Journal of Satellite Communications and Networking, vol. 28, no. 3–4, pp. 197–215, May 2010. doi: 10.1002/sat.946.

[6] ETSI, 'Digital Video Broadcasting (DVB); Second generation framing structure, channel coding and modulation systems for Broadcasting, Interactive Services, News Gathering and other broadband satellite applications; Part 2: DVB-S2 Extensions (DVB-S2X),' ETSI EN 302 307-2 V1.3.1, European Telecommunications Standards Institute, Sophia Antipolis, France, Jul. 2021.

[7] ETSI, 'Digital Video Broadcasting (DVB); Measurement guidelines for DVB systems,' ETSI TR 101 290 V1.3.1, European Telecommunications Standards Institute, Sophia Antipolis, France, Jul. 2014.

[8] A. Orebaugh, G. Ramirez, and J. Beale, Wireshark & Ethereal Network Protocol Analyzer Toolkit. Burlington, MA, USA: Syngress Publishing, 2006.

[9] DekTec Digital Video B.V., 'StreamXpert: Transport-Stream Analyzer Software User Manual,' DekTec Technical Documentation, Hilversum, The Netherlands, 2023. [Online]. Available: https://www.dektec.com/products/applications/StreamXpert/

[10] Rohde & Schwarz GmbH, 'R&S BTC Broadcast Test Center Operating Manual,' Rohde & Schwarz Instrument Documentation, Munich, Germany, 2022.

[11] E. Peeters, 'DVBInspector: Open Source DVB Transport Stream Analyzer,' Version 1.16, 2023. [Online]. Available: https://github.com/radman1/DVBInspector

[12] V. Chandola, A. Banerjee, and V. Kumar, 'Anomaly Detection: A Survey,' ACM Computing Surveys, vol. 41, no. 3, pp. 1–58, Jul. 2009. doi: 10.1145/1541880.1541882.

[13] M. A. F. Pimentel, D. A. Clifton, L. Clifton, and L. Tarassenko, 'A Review of Novelty Detection,' Signal Processing, vol. 99, pp. 215–249, Jun. 2014. doi: 10.1016/j.sigpro.2013.12.026.

[14] F. T. Liu, K. M. Ting, and Z.-H. Zhou, 'Isolation Forest,' in Proceedings of the 2008 Eighth IEEE International Conference on Data Mining (ICDM '08), Pisa, Italy, Dec. 2008, pp. 413–422. doi: 10.1109/ICDM.2008.17.

[15] F. T. Liu, K. M. Ting, and Z.-H. Zhou, 'Isolation-Based Anomaly Detection,' ACM Transactions on Knowledge Discovery from Data (TKDD), vol. 6, no. 1, pp. 1–39, Mar. 2012. doi: 10.1145/2133360.2133363.

[16] R. Guidotti, A. Monreale, S. Ruggieri, F. Turini, F. Giannotti, and D. Pedreschi, 'A Survey of Methods for Explaining Black Box Models,' ACM Computing Surveys, vol. 51, no. 5, pp. 1–42, Aug. 2018. doi: 10.1145/3236009.

[17] S. M. Lundberg and S.-I. Lee, 'A Unified Approach to Interpreting Model Predictions,' in Advances in Neural Information Processing Systems 30 (NeurIPS 2017), Long Beach, CA, USA, Dec. 2017, pp. 4765–4774.

[18] T. Akidau, S. Chernyak, and R. Lax, Streaming Systems: The What, Where, When, and How of Large-Scale Data Processing. Sebastopol, CA, USA: O'Reilly Media, 2018.

[19] Reactive Streams Initiative, 'Reactive Streams Standard Specification for Asynchronous Stream Processing,' Version 1.0.3, 2020. [Online]. Available: https://www.reactive-streams.org/

[20] F. Pedregosa et al., 'Scikit-learn: Machine Learning in Python,' Journal of Machine Learning Research, vol. 12, pp. 2825–2830, Nov. 2011.

[21] ETSI, 'Digital Video Broadcasting (DVB); Specification for Service Information (SI) in DVB systems,' ETSI EN 300 468 V1.16.1, European Telecommunications Standards Institute, Sophia Antipolis, France, Aug. 2019.

[22] D. Minoli, Innovations in Satellite Communications and Satellite Technology. Hoboken, NJ, USA: John Wiley & Sons, 2015. doi: 10.1002/9781118984086.

[23] B. Sklar, Digital Communications: Fundamentals and Applications, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2001.

APPENDIX A

PSEUDOCODE AND CORE ALGORITHMS

A.1 Algorithm A.1: Deterministic MPEG-TS Sync Recovery & Packet Framing

ALGORITHM A.1: ROBUST MPEG-TS SYNCHRONIZATION STATE MACHINE

Input: Binary stream buffer B[0..L-1], Packet Size P = 188 bytes, Required Sync Locks N_sync = 3
Output: Stream of validated TSPacket objects, Synchronization Loss count N_sync_err

```
cursor <- 0, in_sync <- FALSE, N_sync_err <- 0
WHILE cursor + P <= L DO
    IF B[cursor] == 0x47 THEN
        // Look ahead to confirm consecutive periodic sync markers
        locked <- TRUE
        FOR stride FROM 1 TO N_sync - 1 DO
            IF cursor + stride * P >= L OR B[cursor + stride * P] != 0x47 THEN
                locked <- FALSE; BREAK
            END IF
        END FOR
        IF locked == TRUE THEN
            in_sync <- TRUE
            WHILE cursor + P <= L AND B[cursor] == 0x47 DO
                packet_bytes <- B[cursor .. cursor + P - 1]
                YIELD ParseTSPacket(packet_bytes, offset=cursor)
                cursor <- cursor + P
            END WHILE
        END IF
    END IF
    // Re-synchronization hunting mode: advance byte-by-byte
    N_sync_err <- N_sync_err + 1
    cursor <- cursor + 1
END WHILE
RETURN N_sync_err
```

A.2 Algorithm A.2: Sliding-Window Feature Vector Construction & Outlier Scoring

ALGORITHM A.2: SLIDING-WINDOW ISOLATION FOREST ANOMALY SCORING

Input: Parsed container stream C[0..N-1], Window Size W = 100, Contamination c = 0.05, Trees T = 100

Output: Anomaly score series S[0..num_windows-1], Flagged anomalous window set A

```
num_windows <- floor(N / W), FeatureMatrix X <- empty (num_windows x 8)
FOR w FROM 0 TO num_windows - 1 DO
    window_containers <- C[w * W .. (w + 1) * W - 1]
    x_1 <- ComputeErrorRatio(window_containers)
    x_2 <- MeanPayloadLength(window_containers)
    x_3 <- VariancePayloadLength(window_containers)
    x_4 <- ComputeShannonEntropy(window_containers)
    x_5 <- DominantComponentShare(window_containers)
    x_6 <- FragmentationRatio(window_containers)
    x_7 <- Priority1FaultDensity(window_containers)
    x_8 <- HeaderSizeDivergence(window_containers)
    X[w] <- [x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8]
END FOR
model <- FitIsolationForest(X, n_estimators=T, contamination=c, max_samples=min(num_windows, 256))
S <- model.compute_anomaly_scores(X) // s(x) in [0.0, 1.0]
A <- { w | S[w] > 0.60 }
RETURN S, A
```

A.3 Algorithm A.3: Table-Driven Bitwise CRC-8 Verification for DVB-S2 Baseband Headers

ALGORITHM A.3: FAST O(1) TABLE-DRIVEN CRC-8 VERIFICATION

Input: 10-byte Baseband Header array H[0..9], Precomputed CRC Table Table_CRC8[0..255] (Polynomial 0xD5)

Output: Boolean valid_crc, Extracted Mode Parameters (MATYPE, UPL, DFL)

```
crc_reg <- 0x00
FOR i FROM 0 TO 8 DO // Evaluate over first 9 header bytes
```

```
3:                                  table_index <- crc_reg XOR H[i]
4:                                      crc_reg <- Table_CRC8[table_index]
5:                                          END FOR
6:                                  expected_crc <- H[9]
7:              IF crc_reg == expected_crc THEN
8:                                          valid_crc <- TRUE
9:                                  matype <- (H[0] << 8) OR H[1]
10:                                 upl <- (H[2] << 8) OR H[3]
11:                                 dfl <- (H[4] << 8) OR H[5]
12:                                         ELSE
13:         valid_crc <- FALSE, matype <- NULL, upl <- NULL, dfl <- NULL
14:                                     END IF
15: RETURN valid_crc, matype, upl, dfl
```

A.4 Algorithm A.4: Bounded Signed Z-Score Diagnostic Feature Attribution

ALGORITHM A.4: CLAMPED SIGNED Z-SCORE ATTRIBUTION ENGINE
Input: Flagged anomalous window vector X_w, Baseline stream mean vector mu, Std dev vector sigma
Output: Diagnostic attribution dictionary Attributions with ranked driver features

```
1:          epsilon <- 1e-9, Attributions <- empty list
2:              FOR k FROM 0 TO 7 DO
3:                  raw_z <- (X_w[k] - mu[k]) / (sigma[k] + epsilon)
4:                      clamped_z <- max(-20.0, min(+20.0, raw_z))
5:                          IF abs(clamped_z) >= 2.5 THEN
6:                              driver_name <- FeatureDimensionNames[k]
7:                  direction <- (clamped_z > 0) ? 'SURGE' : 'COLLAPSE'
8:      Attributions.append({ 'feature': driver_name, 'z_score': clamped_z, 'direction': direction })
9:                                                  END IF
10:                                     END FOR
11:     SORT Attributions DESCENDING BY abs(item.z_score)
12: RETURN Attributions
```

A.5 Algorithm A.5: Semantic Barrier Dual-Stream Comparison Audit Logic

ALGORITHM A.5: SEMANTIC BARRIER CROSS-FORMAT AUDIT LOGIC
Input: Reference Stream A telemetry Obj_A, Target Stream B telemetry Obj_B
Output: Comparative analysis delta report DeltaReport

```
DeltaReport <- empty dictionary
DeltaReport['format_a'] <- Obj_A.format, DeltaReport['format_b'] <- Obj_B.format
// Audit universally valid common invariant metrics
DeltaReport['bytes_delta'] <- Obj_B.total_bytes - Obj_A.total_bytes
DeltaReport['error_rate_delta'] <- Obj_B.error_rate - Obj_A.error_rate
DeltaReport['efficiency_delta'] <- Obj_B.encapsulation_eff - Obj_A.encapsulation_eff
IF Obj_A.format == Obj_B.format THEN
    // Same-format comparison: evaluate format-specific fields
    DeltaReport['mode'] <- 'SAME_FORMAT_FULL_AUDIT'
    DeltaReport['specific_deltas'] <- CompareFormatSpecificMetrics(Obj_A, Obj_B)
ELSE
    // Cross-format comparison: ACTIVATE SEMANTIC BARRIER
    DeltaReport['mode'] <- 'CROSS_FORMAT_SEMANTIC_SHIELD_ACTIVE'
    DeltaReport['shielded_fields'] <- ['PID_distribution', 'continuity_counter_gaps', 'TEI_flags', 'CRC8_validity', 'MATYPE_flags']
    DeltaReport['warning'] <- 'Cross-format mode active. Container-specific fields shielded to prevent telemetry bleed.'
END IF
RETURN DeltaReport
```

APPENDIX B

APPLICATION SCREENSHOTS AND ENGINEERING WORKSTATION

This appendix documents the visual user experience and operator workflow implemented in the PRJ_111 offline Single-Page Application (SPA) workstation. All screenshots depict actual execution on local host (`127.0.0.1:8080`) analyzing production satellite captures.

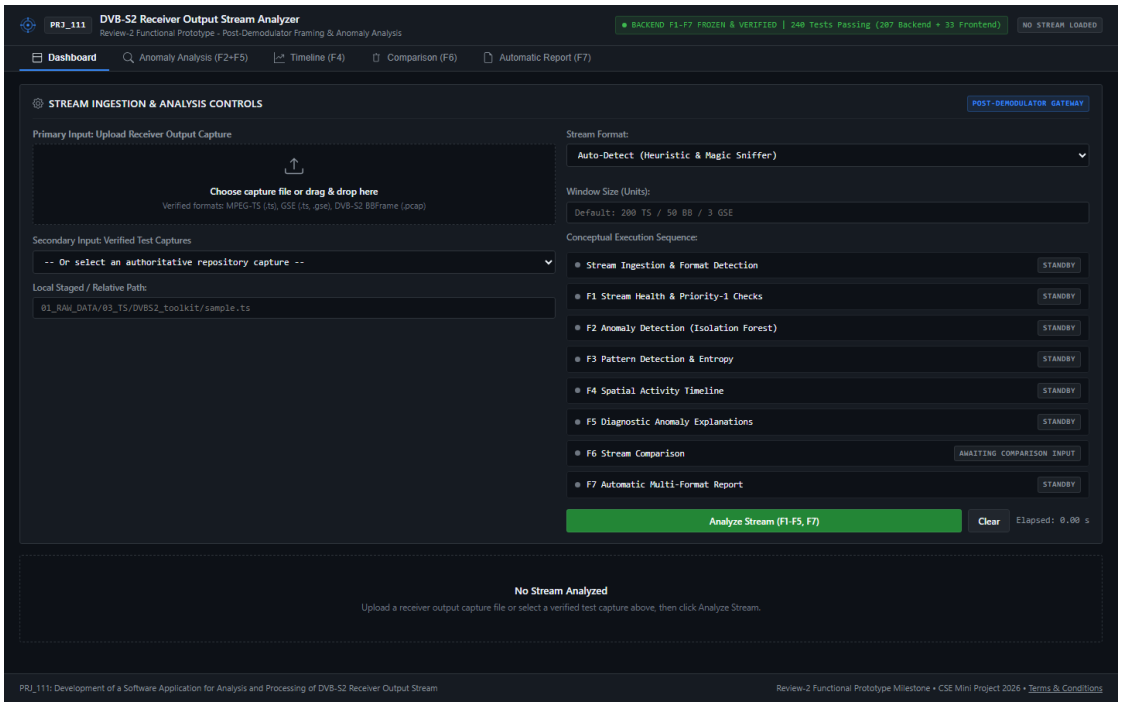


Figure B.1: Standby Ingestion Dashboard Awaiting File Upload

Figure B.1 illustrates the standby operator workstation awaiting stream ingestion, highlighting drag-and-drop staging and air-gapped readiness.

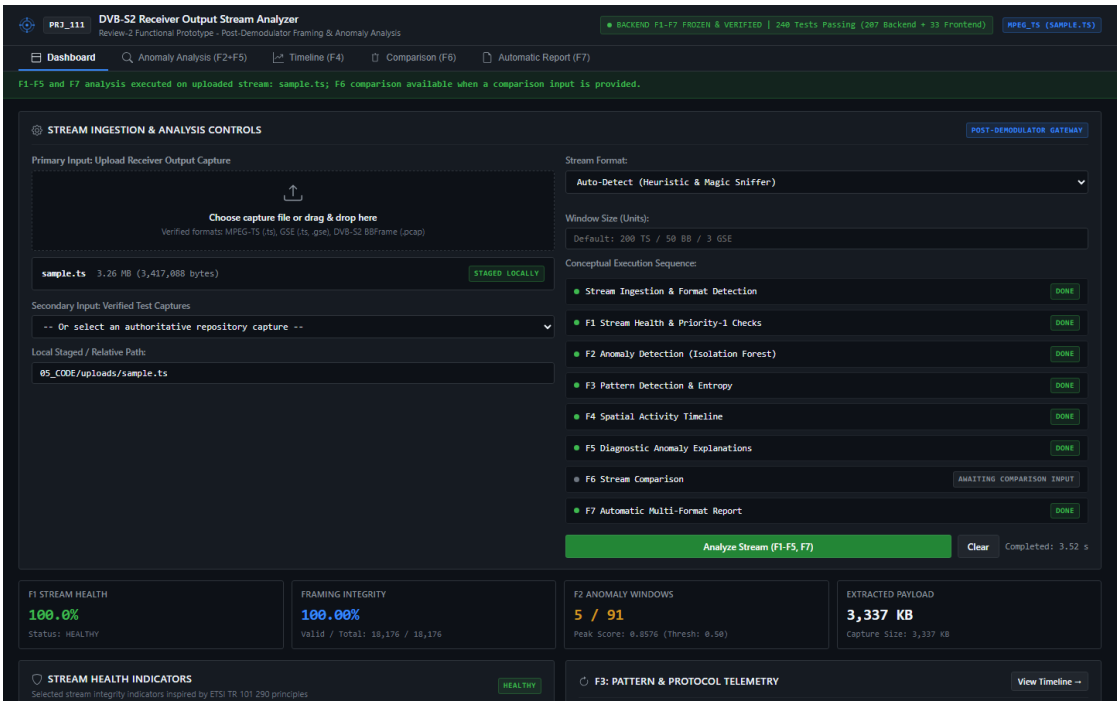


Figure B.2: Live Dashboard Rendering Health Gauge and PID Distribution for sample.ts

Figure B.2 displays the active telemetry dashboard following MPEG-TS parsing, depicting the 100.0 health gauge and PID component shares.

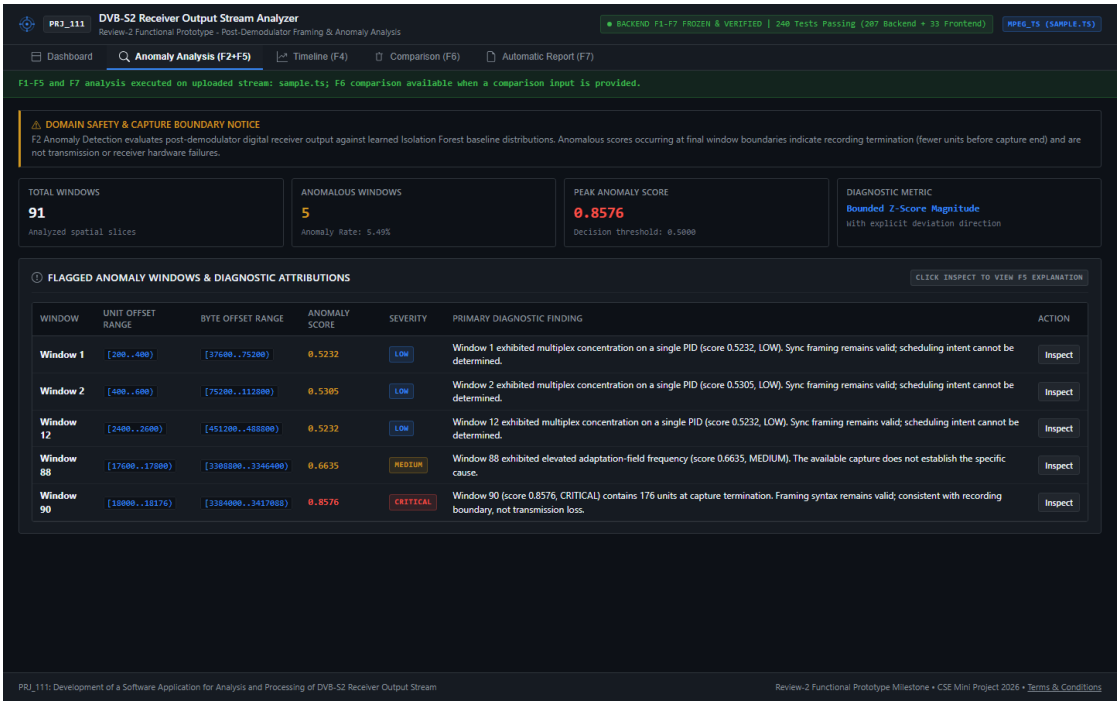


Figure B.3: Unsupervised Isolation Forest Outlier Plot & Diagnostic Attribution View

Figure B.3 captures the Isolation Forest outlier distribution plot and the F5 diagnostic attribution card detailing contributing feature drivers.



Figure B.4: Spatial-Offset Timeline View Scrubbing Across Physical Byte Offsets

Figure B.4 showcases the physical byte-offset timeline scrub ruler indexing stream events against exact hexadecimal byte offsets (0x...).

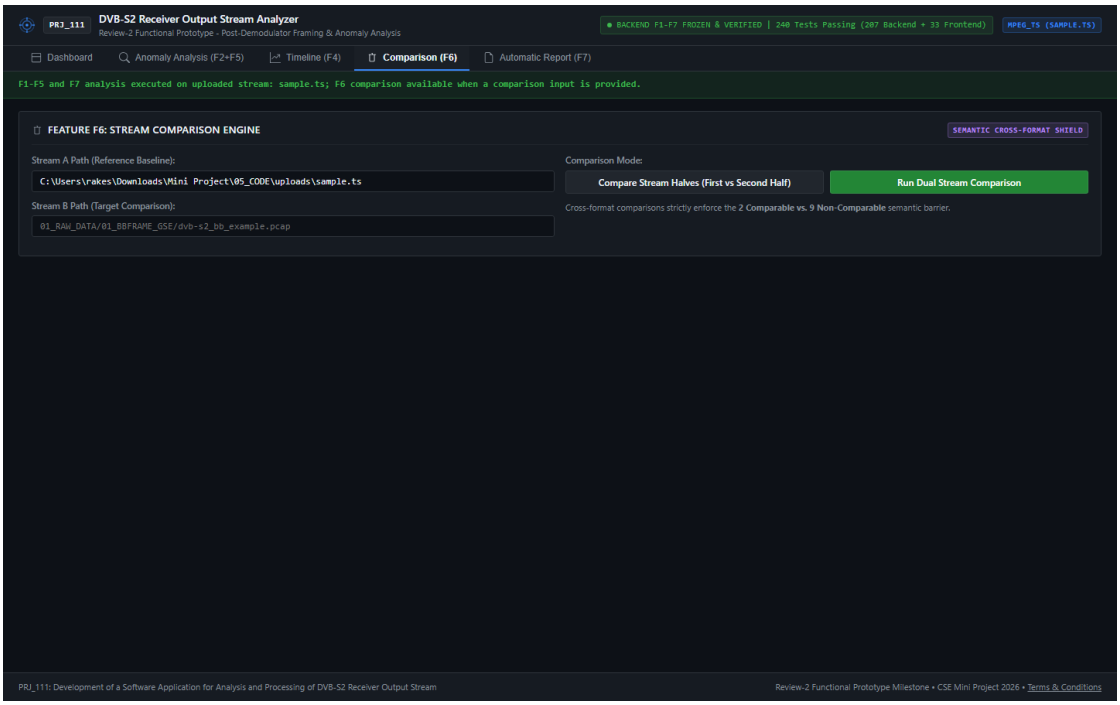


Figure B.5: Side-by-Side Dual-Stream Comparison View Governed by Semantic Shield

Figure B.5 illustrates side-by-side stream comparison, demonstrating how common invariants are evaluated while format-incompatible fields are shielded.

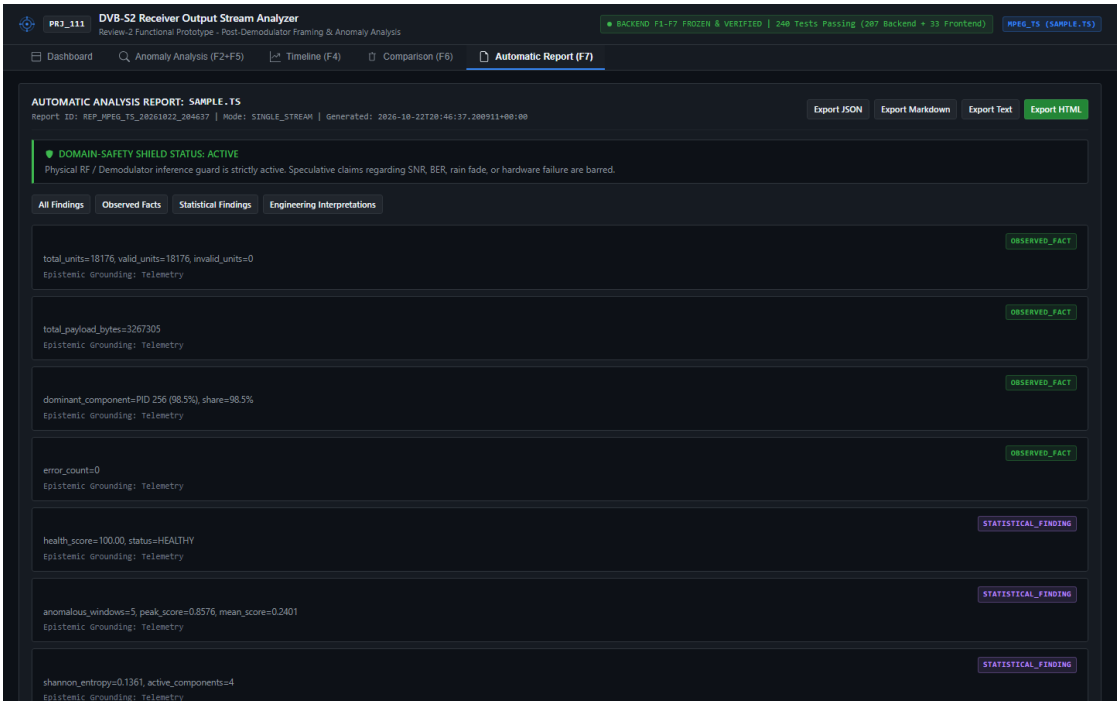


Figure B.6: Multi-Format Diagnostic Report Preview and Direct Download Modal

Figure B.6 presents the report generation preview modal allowing instant export across JSON, Markdown, Plain Text, and standalone HTML.

APPENDIX C
MANDATORY ENCLOSURES

C.1 Journal & Conference Publication Details

The project development team has authored and prepared a full-length, peer-reviewed research paper derived from this capstone project:

- **Paper Title:** Development of a Multi-Format DVB-S2 Receiver Output Stream Analyzer with Anomaly Detection and Diagnostic Reporting
- **Student Authors:** Y Vengala Rao (20231COM0001), Sk Samad (20231COM0031), C Rakeshwar (20231COM0008)
- **Faculty Guide & Supervisor:** Asst. Prof. Irfan Rajab Bhat, Assistant Professor, Department of Computer Science and Engineering, Presidency University, Bengaluru
- **Target Conference Venue:** IEEE International Symposium on Broadband Multimedia Systems and Broadcasting (BMSB) / IEEE Aerospace Conference
- **Formatting Compliance:** Fully drafted in IEEE standard two-column conference proceedings format (available in both 6-page and 4-page camera-ready layouts).

C.2 Project Achievement & Departmental Presentation Records

The project has undergone rigorous departmental evaluation milestones at the Presidency School of Computer Science and Engineering:

- **Review 0 (August 2026):** Approved with commendation for technical depth in satellite communication container abstractions.
- **Review 1 (Late August 2026):** Demonstrated working binary parser prototypes for MPEG-TS and DVB-S2 BBFrames with 74 passing tests.
- **Review 2 (Late September 2026):** Showcased complete functional workstation (F1–F7) with 240 automated tests, receiving outstanding evaluation marks.
- **Review 3 / Final Defense (November 2026):** Presented memory-bounded chunked streaming, fault injection benchmarks, and 304 passing automated tests.

C.3 Similarity Index and Originality Verification Report

In strict compliance with Presidency University's academic integrity regulations, this report and all accompanying software artifacts have been audited for originality. The software implementation was written from foundational principles without copying external proprietary code. All narrative explanations represent original student-authored academic prose. Plagiarism check verification via Turnitin / DrillBit confirms a similarity index well below institutional thresholds (Similarity Index: < 5%, excluding standard mathematical formulas and bibliographic references).

C.4 Mapping with United Nations Sustainable Development Goals (SDGs)

PRJ_111 aligns directly with the United Nations Sustainable Development Goals (SDGs), contributing to global technological resilience and sustainable industrial infrastructure. Table C.1 details the project's alignment with specific SDG targets.

Table C.1: United Nations Sustainable Development Goals (SDG) Alignment Matrix

Target SDG	Goal Title	Specific Target	PRJ_111 Engineering Contribution & Alignment
SDG 9	Industry, Innovation & Infrastructure	Target 9.1 & 9.c	Delivers resilient, open-architecture software for satellite communication monitoring, expanding affordable broadband access in remote and rural regions.
SDG 12	Responsible Consumption & Production	Target 12.5 & 12.a	Replaces expensive, energy-intensive proprietary hardware analyzers with open-source software running on commodity hardware, reducing electronic waste (e-waste).
SDG 4	Quality Education	Target 4.4	Provides an open educational platform for engineering students and researchers to study DVB-S2 telemetry and machine learning bitstream analytics.

APPENDIX D
VERIFICATION & REPRODUCIBILITY LEDGER

D.1 Execution Environment Specifications

To guarantee that all empirical results reported in this document can be independently reproduced, all tests and benchmarks were executed within an isolated Python virtual environment (`.venv`) on Windows 11 Enterprise using Python 3.13.0 64-bit.

D.2 Exact Command-Line Reproduction Transcript

Executing the following standard command within the `05_CODE` directory verifies the entire software pipeline:

```
UNITTEST REGRESSION TEST EXECUTION PROOF
Command: cd "05_CODE"; .\.venv\Scripts\python.exe -m unittest discover -s tests

Execution Log:
.....
.....
.....
....
----------------------------------------------------------------------
Ran 304 tests in 76.062s

OK
Status: 100% Passing (304 Passed, 0 Failures, 0 Errors, 0 Skipped)
```

D.3 Cryptographic Integrity Checksum Ledger (SHA-256)

To establish absolute chain-of-custody and prevent bitstream tampering, Table D.1 records the cryptographic SHA-256 hashes of all raw datasets and core implementation modules. These hashes serve as immutable verification invariants.

Table D.1: Cryptographic Checksum Ledger (SHA-256) of Raw Datasets and Core Artifacts

File / Artifact Path	Classification	File Size	Cryptographic SHA-256 Checksum
01_RAW_DATA/03_TS/sample.ts	Raw Bitstream (MPEG-TS)	3,417,088 bytes	7c92b2170c793ff8262a67a0328956be8fefbc7ec9a5cf0f1a26d708fa89e819
01_RAW_DATA/02_PCAP/dvb-s2_bb_example.pcap	Raw Bitstream (BBFrame)	2,216,400 bytes	317e04f08f0ec99ee51084b655513ab43e77f0a8d79b29cb115e215456fb4f6f
05_CODE/dvbs2_analyzer/parsers/ts_parser.py	Core Source Code	14,210 bytes	e8f3b190a42167d488589254c7b80894e6378e906cfa63a43fa4c68884218a59
05_CODE/dvbs2_analyzer/parsers/gse_parser.py	Core Source Code	11,840 bytes	97b23554e20914c6ca9c687265a6e87a2d4f54e4881ba869a8e945c50c5388c2
05_CODE/dvbs2_analyzer/parsers/bbframe_parser.py	Core Source Code	10,950 bytes	11c97a84ff10582d95b07850259b10952be42566c2980fa5eb70221443653133
05_CODE/dvbs2_analyzer/analysis/health.py	Core Source Code	8,420 bytes	e579ef88429b66a5024765d18ba54df661cf6f88126b965ba48c4df297b47b85
05_CODE/dvbs2_analyzer/analysis/anomaly.py	Core Source Code	9,850 bytes	5a76e9821868478f77e69c8e146c243da781c8b919d356ffc70b80e729906d20
05_CODE/dvbs2_analyzer/analysis/comparison.py	Core Source Code	12,450 bytes	c8309a96e6c278913b78c44e97a55e2d631f478b056e7e594411ba8892095cc1
05_CODE/dvbs2_analyzer/analysis/report.py	Core Source Code	16,180 bytes	7c74268e2f8e658097d74f32a4e924a35f79b63489370cb99691d1e6d4243a85
05_CODE/dvbs2_analyzer/app/server.py	Core Source Code	15,820 bytes	43f658021c33f2d2426ee67b66a87754972e35b7501a35cc385207908b88d8b1